zkAgent: Verifiable LLM Agent Execution via One-Shot Transcript Proofs

Lizheng Wang, Hancheng Lou, Chongrong Li, Yu Yu, and Yuncong Hu*

Shanghai Jiao Tong University, Shanghai, China
{lizhengwang,louhancheng,chongrongli,yyuu,huyuncong}@sjtu.edu.cn

Abstract. LLM-based agents, which interleave large language model inference with external tool calls, are increasingly deployed in high-stakes settings. In real-world deployments, both model inference and provider-hosted tool execution occur behind provider APIs and are therefore opaque to users. This opacity creates an execution-integrity gap: a malicious provider may substitute the advertised model or fabricate tool observations to steer subsequent agent behavior. Existing zero-knowledge proof systems for LLMs prove only the Transformer computation of a single token, leaving the rest of the inference pipeline, long-form autoregressive generation, and external tool interactions outside the proof.

We present zkAgent, the first SNARK system for *verifiable agent execution*. zkAgent proves the complete inference pipeline, from token-to-embedding lookup and positional encoding to Transformer computation and decoding. It further binds each tool observation to an authenticated execution via zkTLS or zkVM subproofs, yielding a single end-to-end proof. To scale beyond per-token proving, we introduce *one-shot transcript proving*: by exploiting the Transformer's causal attention mask, zkAgent proves an entire multi-step agent transcript in a single forward pass, avoiding the substantial overhead incurred by one-proof-per-token generation. We make this batched proof sound with a weight-dependent quantization scheme that is both input-independent and unconditionally complete.

On GPT-2 with a 512-token transcript, zkAgent achieves a 1,047.4× prover speedup over the state of the art (zkGPT, USENIX Security '25), amortizing to 0.30 s/token. On a real-world *coding-assistant* execution, zkAgent completes end-to-end proving in 93.18 s with 1.63 s verification, making verifiable agent execution practical.

1 Introduction

Large language model (LLM)-based agents [41] interleave model inference with external tool use to act on users' behalf. In production, such agents have rapidly moved beyond conversational interfaces: coding assistants can modify software repositories [3], while general-purpose agents browse and act on the open web [52]. Emerging standards now address cross-organizational agent discovery and trust (e.g., ERC-8004 [19]), and agent-mediated purchases (e.g., the Agentic Commerce

* Corresponding author.

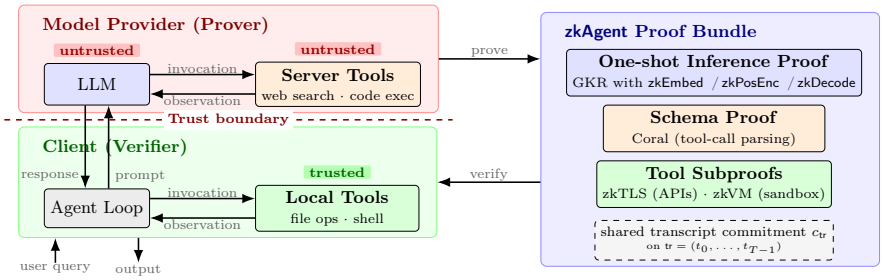


Fig. 1: zkAgent architecture and proof bundle. The *model provider* (prover) hosts the LLM and server-side tools (web search, code execution), both above the trust boundary and requiring cryptographic proof; the client (verifier) runs the agent loop, which calls the provider for model inference and may additionally invoke trusted local tools (e.g., file or shell operations).

Protocol [56]). As agent outputs increasingly trigger changes to external systems and assets, verifying the integrity of the underlying model inference and tool execution becomes a security-critical requirement.

In modern agent deployments, each LLM invocation executes behind the provider's API, along with an increasingly rich set of server-hosted tools (e.g., web search and code execution in managed sandboxes [6]) bundled into that API. Even when the agent loop runs entirely on the user's device, these provider-executed steps remain opaque to the user, as illustrated in Fig. 1. This opacity creates an *end-to-end execution-integrity gap* [43]: an untrusted provider may deviate arbitrarily from the prescribed agent procedure: (i) by silently substituting the advertised model with a cheaper or backdoored alternative; or (ii) by fabricating *tool observations*, such as forging web-search results or misreporting sandbox outputs.

Recent works on *zero-knowledge proofs for large language models* [69,60,13,63] (zkLLM) have demonstrated the feasibility of verifiable outsourced LLM inference. These systems encode core LLM computations as an arithmetic circuit and leverage SNARK techniques [16,29,9,7,30] to generate succinct proofs that the committed model was faithfully executed on a given input. Despite recent progress, existing zkLLM approaches remain insufficient for verifiable agent execution. In particular, they focus primarily on computations within Transformer blocks, and therefore do not capture the full inference pipeline from input tokens to decoded output tokens. Moreover, they do not support multi-round agent executions that interleave model inference with external tool use. Closing these gaps poses several fundamental challenges.

Challenge 1: Proving complete inference beyond Transformer blocks. Completing the full inference pipeline requires proving three components generally omitted by prior zkLLM systems: token-to-embedding lookup [73,48], positional encoding [67], and logit-to-token decoding [34]. Each poses a distinct proving challenge. First, embedding lookup maps each input token to a high-dimensional

vector via a vocabulary-sized table, which can contain tens of millions of entries even for GPT-2 [61]. Direct applications of generic lookup arguments [28,32] and offline memory-checking techniques [66,65] incur proving overhead linear in the table size, making online proving impractical at this scale. Second, positional encoding injects position information through a position-dependent linear transformation. RoPE [67] and its extensions [59,23] are widely used in modern LLMs [71,80,20], but a direct arithmetization [69] compiles each rotation into a per-token $d_h \times d_h$ matrix–vector multiplication, incurring $\Theta(Td_h^2)$ overhead per head even with time-optimal matmul protocols [70]. Finally, decoding maps a logit vector to the next output token according to a public rule such as top-k sampling [34]. Beyond proving correct candidate-set extraction and correct Softmax-distribution computation, the prover must additionally show that the output token was *drawn from this distribution at the correct probability*. No existing proof system provides a construction for this complex random sampling.

Challenge 2: Proving long-form generation efficiently. Even for a single query, LLM may generate a multi-token continuation over many autoregressive steps, each requiring a forward pass conditioned on all preceding tokens. Thus, a proof of LLM inference must ensure the correctness of each autoregressive step. For a 482-token continuation on GPT-2, stepwise proving with zkGPT [60] requires over 41 hours and produces proofs totaling $\sim$934 MiB. Recursive proof composition [72,17,9] can compress the resulting proof size but does not reduce the dominant stepwise prover work, and folding-based batching [39,38] still requires prover work proportional to the total cost of all forward passes. Although it may seem natural to evaluate the model once on the full transcript and prove all token predictions in a single pass, doing so without additional constraints may allow future tokens to influence earlier predictions, violating the causal structure of autoregressive generation. Enforcing causal dependence within a single batched proof while guaranteeing equivalence to stepwise autoregressive generation remains an open problem in prior zkLLM work [69,60,77,13]. In quantized inference, such equivalence further requires input-independent per-layer scales so that corresponding prefix computations in the batched and stepwise executions are quantized identically. Prior zkML quantization schemes do not simultaneously ensure input independence and completeness: data-dependent dynamic schemes [44] use input-specific scales, whereas fixed calibration-based schemes [77] may reject valid executions whose activations exceed the calibrated range.

Challenge 3: Proving external tool interactions. Unlike standalone LLM inference, an agent execution interleaves model-generated content with *externally injected* tool observations (e.g., web search results, APIs responses, or sandbox outputs). Once incorporated into the model context, these observations can directly influence subsequent model outputs and agent actions. Existing zkLLM systems [69,60,13,77] do not model these tool interactions and therefore establish neither the integrity of the observations nor their binding to subsequent model inference. This leaves a critical gap: a malicious provider could tamper with tool results or fabricate plausible-looking tool traces while still producing a seemingly

coherent final response [43]. Closing this gap requires binding tool observations into the proof—authenticating responses against the claimed endpoints, proving the correctness of deterministic sandbox computations, and composing these guarantees with LLM-generation correctness into a single end-to-end statement.

1.1 Our Contributions

To address these challenges, we present zkAgent, a zero-knowledge proof system for end-to-end agent executions with tool use. We give an overview of its architecture in Fig. 1.

Complete inference proof. We design dedicated protocols for each component omitted by prior zkLLM systems, thereby providing a complete inference proof from input tokens to decoded output. For token-to-embedding lookup, we present zkEmbed, a preprocessing-based *matrix-lookup* protocol that exploits the highly rectangular structure of embedding tables in modern LLMs [61,80,71]. After one-time preprocessing on the fixed table, it achieves the online proving cost independent of the vocabulary size. zkPosEnc provides efficient positional-encoding proofs; for RoPE [67], it exploits the structural sparsity of the rotation matrix to reduce overhead from $O(Td_h^2)$ to $O(Td_h)$ over prior method [69]. zkDecode provides efficient decoding proofs; for top-k sampling, it introduces a *verifiable random sampling* subprotocol based on inverse-CDF selection with a coin deterministically derived from the committed prefix, achieving asymptotically optimal proving cost.

One-shot transcript proving via causal masking. We introduce *one-shot transcript proving*, which verifies all model-generated tokens with a single proof of a causally masked forward pass rather than one proof per token. The construction relies on the fact that, in a decoder-only LLM, self-attention is the only sublayer that mixes information across positions. By applying a lower-triangular *causal attention mask* [73] at every attention layer, we enforce autoregressive causality within a single batched forward pass: each position t attends only to positions $\leq t$, so its logits depend only on preceding tokens and all logits can be computed in parallel in one full-transcript pass. We also formally prove that this one-shot causally-masked forward pass yields the same per-position logits as stepwise autoregressive generation—a property we call *prefix equivalence* (Appendix E). To preserve prefix equivalence in quantized arithmetic, we design *weight-dependent quantization*, deriving each layer's scale from a worst-case bound determined solely by the committed model parameters rather than the input (§4.1). The resulting scales are input-independent and ensure completeness for honest executions. For tool-interleaved transcripts, we further introduce a transcript-derived *generation mask* that restricts next-token consistency checks to model-generated positions. To our knowledge, zkAgent is the first work to formally establish prefix equivalence and to design a protocol that avoids the one-proof-per-token structure, enabling long, multi-round agent traces to be proved via a single forward-pass proof.

Verifiable tool observations and execution. We design tool-specific protocols that bind each tool invocation and observation into the transcript proof, covering the two dominant categories of provider-hosted tools: remote API calls and

sandboxed code execution. For remote API calls, we incorporate zkTLS-style attestations [79], which cryptographically bind each recorded response to its corresponding request and the identity of the target server. For code execution, we support the widely adopted Python sandbox [54,4] by building on existing zkVM techniques [68] to prove execution correctness. Beyond binding tool *outputs*, we also enforce the *well-formedness* of tool *invocations*: we prove that any purported tool call conforms to a public invocation schema [55,5,76] via provable parsing techniques such as Coral [2], and link each observation back to its invocation.

Prototype and evaluation. We implement zkAgent in C++ and evaluate it on GPT-2 with two interchangeable lookup backends, Shout [65] and LogUp [32,58]. Given a 30-token prompt and a final transcript length of $T = 512$ (482 generated tokens), zkAgent(Shout) proves the entire transcript in a single shot in 142.45 s, amortizing to 0.30 s per generated token, while the succinct verifier takes only 2.63 s for the entire transcript. This yields a 1,047.4× prover-time and 1,657.5× verifier-time reduction over the state-of-the-art zkGPT baseline [60](149,192.60 s and 4,361.09 s, respectively). zkAgent(LogUp) offers a complementary tradeoff, shrinking the proof to 3.84 MiB and verification to 1.50 s at 1.4× slower proving against zkAgent(Shout), still much faster than zkGPT.

Microbenchmarks further validate the efficiency of the additional components needed for *complete* inference beyond Transformer blocks: After a one-time preprocessing step taking 942.43 s and requiring 4.52 GiB of auxiliary storage,zkEmbed proves a batch of 1,024 embedding queries online in 0.27 s; zkPosEnc reduces proving time by 15.7×–30.1× versus the naive matmul arithmetization used in zkLLM [69]; zkDecode amortizes greedy decoding to 0.033 s/token and top-k to 0.15 s/token of prover overhead.

Finally, we evaluate zkAgent on two representative tool-using agent traces: a 152-token weather-agent trace (zkTLS-attested API call) and a 143-token coding-assistant trace (zkVM-attested sandbox execution). With the Shout backend, zkAgent generates the corresponding end-to-end proof bundles in 95.23 s and 93.18 s, respectively, while verification takes 1.45 s and 1.63 s. In both cases, the inference proof dominates the proving cost; the schema and tool subproofs together add only 7.62 s and 6.18 s, respectively.

2 Preliminaries

2.1 Notations

We use κ to denote the security parameter. Let $\mathbb{F}$ be a prime field of order p such that $\log p = \Omega(\kappa)$, and let $\mathbb{F}_{<D}[X]$ denote the set of univariate polynomials over $\mathbb{F}$ of degree strictly less than D. For integers $a \le b$, we write $[a, b] \coloneqq \{a, \ldots, b\}$ and $[a, b) \coloneqq \{a, \ldots, b-1\}$ for the closed and half-open integer ranges, respectively. In particular, $[n] \coloneqq [0, n) = \{0, \ldots, n-1\}$ for $n \in \mathbb{N}$, and all indices start from 0. We write $\{0,1\}^n$ for the boolean hypercube, use standard asymptotic notation, and write $x \xleftarrow{\$} S$ when x is sampled uniformly from a finite set S. We use bold lowercase letters for vectors (e.g., $\boldsymbol{x} \in \mathbb{F}^n$), bold uppercase letters

for matrices (e.g., $\boldsymbol{W} \in \mathbb{F}^{m\times n}$), and plain letters for scalars. For $\boldsymbol{x} \in \mathbb{F}^n$, x_i denotes its i-th entry, $(x_i)_{i\in[n]}$ the indexed tuple of entries, and $\{x_i\}_{i\in S}$ the multiset of entries indexed by $S \subseteq [n]$; we omit the index set when it is clear from context. For matrices, we use bracket indexing: $\boldsymbol{W}[i,j]$ denotes the (i,j)-entry, $\boldsymbol{W}[i,:] \in \mathbb{F}^n$ its i-th row, and $\boldsymbol{W}[:,j] \in \mathbb{F}^m$ its j-th column. For $\boldsymbol{x},\boldsymbol{y} \in \mathbb{F}^n$, we write $\langle \boldsymbol{x},\boldsymbol{y}\rangle \coloneqq \sum_{i\in[n]} x_i y_i$ for the inner product and $\boldsymbol{x} \circ \boldsymbol{y} \coloneqq (x_i y_i)_{i\in[n]}$ for the Hadamard product. We use $\mathsf{tr} = (t_i)_{i\in[T]} \in [N]^T$ for a token sequence of length T over a vocabulary of size N, with $\mathsf{tr}_{\le k}$ and $\mathsf{tr}_{<k}$ denoting its prefixes $(t_0,\ldots,t_k)$ and $(t_0,\ldots,t_{k-1})$, and $\boldsymbol{H}_\ell \in \mathbb{F}^{T\times d}$ for the hidden-state tensor at the output of layer ℓ.

2.2 Lookup Arguments

Lookup arguments [32] are a cryptographic primitive for proving that a committed vector of query values is drawn from a committed table. Formally, given a committed table $\boldsymbol{b} \in \mathbb{F}^N$ and a committed query vector $\boldsymbol{a} \in \mathbb{F}^\ell$, a lookup argument proves the multiset inclusion $\{a_i\}_{i\in[\ell]} \subseteq \{b_j\}_{j\in[N]}$. This primitive is widely used in zkML [69,60,33] to prove non-arithmetic operations (e.g., $\exp(\cdot)$).

The appropriate lookup strategy depends on the size and structure of the table. For small tables, direct protocols such as LogUp [32] and Shout [65] remain practical even though their online prover costs scale with N. Papini et al. [58] further use GKR to verify the LogUp identity without committing to the reciprocal polynomial. For large structured tables, Lasso-style decomposition [66] reduces a lookup over the full table to lookups over smaller subtables derived from the original query, making the prover cost depend on the decomposed representation rather than the full table size. For large unstructured tables, preprocessing-based lookup arguments such as cached quotients (cq) [24] and Baloo [82] shift the N-dependent work to a one-time preprocessing phase, so that the online prover cost scales with ℓ rather than N. zkAgent combines all three strategies: for large structured tables, we first applies Lasso-style decomposition to reduce each lookup to lookups over smaller subtables, and then proves the resulting lookup relations using GKR-optimized LogUp and Shout (§5.1). For the large unstructured vocabulary-sized embedding table, our zkEmbed protocol instead uses preprocessing based on cq (Appendix D).

2.3 GKR Protocol for ML

The GKR protocol [30] is an interactive proof for layered arithmetic-circuit evaluation. Unlike SNARKs such as HyperPlonk [14] and Spartan [64], which commit to the full execution trace, GKR iteratively reduces a claim at one layer to a claim at the preceding layer via the sumcheck protocol [46], continuing until it reaches the input layer. The prover therefore commits only to the input layer and uses a polynomial commitment scheme (PCS) to answer the verifier's resulting random evaluation queries. This feature is particularly valuable in zkML [44,69,60], where the forward-pass trace is too large to commit to directly.

In zkAgent, we likewise adopt GKR, handling arithmetic operations directly inside GKR [44] and non-arithmetic operations via lookup arguments [66,69].

Multilinear extension. A polynomial $p \in \mathbb{F}[X_0, \ldots, X_{n-1}]$ is *multilinear* if it has degree at most 1 in each variable. For any $f : \{0,1\}^n \to \mathbb{F}$, there is a unique multilinear polynomial $\tilde{f} : \mathbb{F}^n \to \mathbb{F}$ such that $\tilde{f}(\boldsymbol{x}) = f(\boldsymbol{x})$ for all $\boldsymbol{x} \in \{0,1\}^n$; this $\tilde{f}$ is the *multilinear extension* (MLE) of f, and can be written as $\tilde{f}(\boldsymbol{X}) = \sum_{\boldsymbol{x}\in\{0,1\}^n} f(\boldsymbol{x}) \cdot \widetilde{eq}(\boldsymbol{x}, \boldsymbol{X})$, where $\widetilde{eq}(\boldsymbol{x}, \boldsymbol{X}) := \prod_{i=0}^{n-1}(x_i X_i + (1 - x_i)(1 - X_i))$.

Sumcheck protocol. For an n-variate polynomial $f : \mathbb{F}^n \to \mathbb{F}$ of bounded individual degree, the sumcheck protocol [46] allows the prover to convince a claim of the form $v = \sum_{\boldsymbol{x}\in\{0,1\}^n} f(\boldsymbol{x})$. It reduces this sum claim to an evaluation claim about f at a random point. A formal description is provided in Appendix C.2.

GKR variant adopted in zkAgent. Classical GKR restricts each gate to reading only from the immediately preceding layer, forcing model weights and residual inputs to be relayed through every intervening layer. We instead adopt the variant of Zhang et al. [84], which permits each gate also read from *any* earlier layer, so that computation layers directly access committed weights and earlier hidden states. To prove non-arithmetic operations within this backend, we additionally use the *advice* pattern [44,60]. For each such operation $y = f(x)$, the prover places both x and y in the committed input layer as advice values. GKR verifies that x matches the value produced by the preceding computation and that y is consumed by the subsequent computation, while a lookup argument verifies that (x, y) is valid for f. The shared input-layer commitment binds the values checked by the lookup argument to those consumed by GKR. For clarity of presentation, we present our construction without the zero-knowledge property; however, this property can be added with only minor overhead using the standard paradigm [78,84]. Formal constructions are given in Appendices C.3 and C.4.

2.4 Quantization for zkML

Since the GKR backend operates over a finite field $\mathbb{F}$, every real-valued activation and weight must first be encoded as a field element. Prior zkML systems [44,13,69,60] follow the fixed-point quantization of Jacob et al. [35]: a real value x is approximated as $x \approx q \cdot s$, where $q \in [-2^Q, 2^Q]$ is a signed integer with magnitude bounded by 2^Q and s is a scale. We embed q into $\mathbb{F}$ via the natural modular map: $q \geq 0$ is represented by q, and $q < 0$ by $p+q$. Since $|q| \ll |\mathbb{F}|/2$, this representation preserves signed arithmetic as long as all intermediate magnitudes stay below $|\mathbb{F}|/2$. Matrix multiplications and additions accumulate into wider integers, so after every layer a *rescale* step that maps the accumulator back to magnitude $\leq 2^Q$ under a layer-specific output scale $S_{\text{out}}^{(l)}$. Appendix F gives the full protocol.

2.5 LLM Inference Pipeline

A decoder-only LLM first applies a public tokenizer to the input prompt to obtain a sequence of token IDs. Each autoregressive step then comprises four

stages: (i) token-to-embedding lookup, (ii) positional encoding, (iii) a stack of Transformer blocks, and (iv) an output head plus a public decoding rule to select the next token, which is appended to the context for the next step. On an input sequence of length T, the forward pass produces a logit matrix in $\mathbb{F}^{T\times N}$ with one row per position, but sequential generation decodes only the last row to obtain the next token. Since tokenization is public, we exclude it from the proved computation and start from the token IDs. Prior zkLLM proof systems [69,60,77] cover only stage (iii), leaving stages (i), (ii), and (iv) unproved. In contrast, zkAgent proves the complete post-tokenization pipeline comprising stages (i)–(iv). Appendix B gives the detailed inference pipeline.

2.6 System and Threat Model

An LLM agent couples a language model with external tools and coordinates their interaction in a loop. Starting from the user input, the agent repeatedly (i) invokes the LLM on the current context, (ii) executes any tools requested by the model, and (iii) appends the resulting tool observations to the context. The loop terminates when the model produces a final response. We represent the resulting execution as a growing *transcript* tr, namely the concatenation of the user input, model outputs, and tool observations.

Threat model. We consider LLM agents operated by a *model provider* that performs LLM inference and hosts server-side tools such as web search and code execution [6,53], as illustrated in Fig. 1. Our model covers two deployment classes. *(i) Fully hosted chat products* (e.g., ChatGPT, Claude.ai [54,6]) execute the entire agent loop on provider-controlled infrastructure. *(ii) API-driven agents* (e.g., Claude Code, Codex CLI [3,51]) execute the orchestration loop on the client, but LLM inference and server-side tools remain hidden behind the provider's API. Tools executed on the client machine (e.g., file or shell operations) are inherently trusted and excluded from the proof. In both cases, the provider holds private model parameters W and publishes a binding commitment c_W. For each query, the client submits an input and receives an execution transcript tr together with a proof bundle. We model the provider as the sole adversary: it may deviate arbitrarily from the prescribed agent procedure, e.g., replacing the committed model with a cheaper or backdoored alternative, or fabricating server-tool observations, such as forged web-search results or misreported sandbox outputs, to steer the subsequent agent behavior.

Out-of-scope attacks. zkAgent targets *execution integrity*; several orthogonal threats are deliberately excluded. *(i) Prompt injection.* Attacks that steer the model through valid-looking inputs [1] are orthogonal to execution integrity: the injected content appears in the transcript and is auditable by the client, but identifying and filtering it is an alignment/policy task rather than a cryptographic one. *(ii) Network-tool semantics.* For network tools, we prove *provenance* rather than the correctness of the server's business logic. Further discussion is deferred to Appendix A.

3 Complete Inference Proof for LLM

As described in Section 2.5, prior zkLLM proof systems cover only the computations within Transformer blocks, leaving three components of the inference pipeline unverified. We close this gap with three dedicated sub-protocols: zkEmbed for token-to-embedding lookup, zkPosEnc for positional encoding, and zkDecode for logit-to-token decoding.

3.1 zkEmbed: Token-to-Embedding Lookup

Let N and d denote the vocabulary size and embedding dimension, respectively. Given a tokenized transcript $\mathsf{tr} = (t_i)_{i\in[T]} \in [N]^T$ and the model's private embedding table $\boldsymbol{W}_E \in \mathbb{F}^{N\times d}$, the prover must show that each row of the embedding output $\boldsymbol{H}_0 \in \mathbb{F}^{T\times d}$ equals the embedding vector indexed by the corresponding token:

$$\forall i \in [T], \quad \boldsymbol{H}_0[i,:] \;=\; \boldsymbol{W}_E[t_i,:]. \tag{1}$$

For the polynomial encoding below, let $\bar{N} := 2^{\lceil \log_2(N+1)\rceil}$ denote the padded table size. We append $\bar{N} - N$ fixed zero rows to $\boldsymbol{W}_E$, obtaining $\boldsymbol{W}_E^{\mathsf{pad}} \in \mathbb{F}^{\bar{N}\times d}$. Since every transcript token satisfies $t_i \in [N]$, this padding leaves the target relation in Eq. (1) unchanged.

The lookup arguments used in prior zkLLM schemes [69,60] can in principle be extended to matrix lookup, but they incur prover overhead linear in the embedding table size Nd (about 2^{25} entries for GPT-2), rather than the number of queries T. In light of this inefficiency, zkEmbed instead builds on the cq preprocessing-based lookup argument [24], shifting all table-size-dependent work to a one-time preprocessing phase and reducing the online prover cost to $O(Td)$, independent of N and $\bar{N}$. At a high level, we reduce the matrix lookup to a scalar lookup via a random linear combination of the d columns. Then, we keep the table side univariate to preserve preprocessing, while expressing the query side over multilinear extensions to enable composition with the downstream GKR proof.

Recap of cq. For a table $\boldsymbol{b} \in \mathbb{F}^{\bar{N}}$ and queries $\boldsymbol{a} \in \mathbb{F}^T$, cq reduces the multiset inclusion $\{a_i\}_{i\in[T]} \subseteq \{b_u\}_{u\in[\bar{N}]}$ to a *logarithmic-derivative* identity [32]. Let m_u denote the multiplicity of b_u in the query multiset. Then the inclusion holds if and only if

$$\sum_{i\in[T]} \frac{1}{a_i + X} \;=\; \sum_{u\in[\bar{N}]} \frac{m_u}{b_u + X}$$

holds as an identity of rational functions in X. After the prover fixes this multiplicity witness with a binding commitment, the verifier samples a uniformly random $\beta \in \mathbb{F}$; by Schwartz–Zippel, except with negligible probability, it suffices to check the identity at $X = \beta$. Let $V = \langle\omega\rangle \subseteq \mathbb{F}^*$ be a multiplicative subgroup of order $\bar{N}$, with vanishing polynomial $Z_V(X) := \prod_{x\in V}(X - x) = X^{\bar{N}} - 1$. Let $\mathcal{T}, M, A \in \mathbb{F}_{<\bar{N}}[X]$ be the polynomials interpolating (b_u), (m_u), and (f_u) over V,

respectively, where $f_u \coloneqq m_u/(b_u+\beta)$ denotes the table-side reciprocal witness. The pointwise identities $f_u(b_u+\beta)=m_u$ hold if and only if $A(X)(\mathcal{T}(X)+\beta)-M(X)$ vanishes on V, or equivalently, if there exists a quotient polynomial $Q(X)$ such that

$$A(X)\cdot\big(\mathcal{T}(X)+\beta\big)-M(X)=Q(X)\cdot Z_V(X). \tag{2}$$

A KZG pairing check succinctly verifies this divisibility relation [37]. Let $\mathsf{Preproc}(\cdot)$ denote the `cq` preprocessing procedure. It outputs the table commitment $c_{\mathcal{T}}$ and, for each $u\in[\bar{N}]$, the commitments $\mathsf{Commit}(L_u)$, $\mathsf{Commit}(\Delta L_u)$, and $c_{Q,u}$. Here L_u is the u-th Lagrange basis polynomial on V and $\Delta L_u(X) \coloneqq (L_u(X)-L_u(0))/X$; Appendix D gives the full construction. By the sparsity of $\boldsymbol{m}$, the online commitments to A and Q are assembled from only those preprocessed components indexed by $\mathsf{supp}(\boldsymbol{m})$, yielding online cost $O(|\,\mathsf{supp}(\boldsymbol{m})|)\le O(T)$ independent of $\bar{N}$.

One-time preprocessing of the embedding table. Since `cq` operates on a scalar table while $\boldsymbol{W}_E$ has d columns, we fold the columns by random linear combination with random challenge $\rho\in\mathbb{F}$, reducing the matrix lookup to a scalar one. However, by Fiat-Shamir [27], ρ must be sampled only after the prover commits to $\boldsymbol{H}_0$, foreclosing direct preprocessing of the folded table. We instead preprocess each column of $\boldsymbol{W}_E$ independently; by the $\mathbb{F}$-linearity of KZG commitments [37], the per-column preprocessed commitments can be aggregated online into those of the folded table once ρ is fixed.

Define the j-th padded embedding column $\boldsymbol{w}_j \coloneqq \boldsymbol{W}_E^{\mathsf{pad}}[:,j]\in\mathbb{F}^{\bar{N}}$ and the public index vector $\mathbf{idx}\coloneqq(0,1,\dots,\bar{N}-1)\in\mathbb{F}^{\bar{N}}$. In a one-time setup for a fixed $\boldsymbol{W}_E$, compute all instances below using the same KZG SRS and the same domain V:

$$\mathsf{pp}_{\mathsf{idx}} \leftarrow \mathsf{Preproc}(\mathbf{idx}), \tag{3}$$

$$\mathsf{pp}_{w_j} \leftarrow \mathsf{Preproc}(\boldsymbol{w}_j), \qquad \text{for } j\in[d]. \tag{4}$$

Online batching: from matrix lookup to scalar lookup. During the online phase, once the prover has committed to $\boldsymbol{H}_0$, the verifier samples the batching challenge $\rho\in\mathbb{F}$. For each table index $u\in[\bar{N}]$ and each query position $i\in[T]$, define the folded scalars

$$b_u \;\coloneqq\; u \;+\; \textstyle\sum_{j\in[d]}\rho^{j+1}\cdot\boldsymbol{W}_E^{\mathsf{pad}}[u,j], \tag{5}$$

$$a_i \;\coloneqq\; t_i \;+\; \textstyle\sum_{j\in[d]}\rho^{j+1}\cdot\boldsymbol{H}_0[i,j]. \tag{6}$$

The constant terms u and t_i bind the claimed row index, and the ρ^{j+1}-weighted sum binds all d entries of the row. It then suffices to prove the scalar multiset inclusion

$$\{a_i\}_{i\in[T]}\subseteq\{b_u\}_{u\in[\bar{N}]}. \tag{7}$$

For soundness, note that if $\boldsymbol{H}_0[i,:]\neq\boldsymbol{W}_E[t_i,:]$ for any i, then $a_i=b_{t_i}$ would require a nonzero degree-$\le d$ polynomial to vanish at ρ, which happens with probability at most $d/|\mathbb{F}|$.

The Lagrange-basis and auxiliary commitments depend only on V and are reused as-is. Let $c_{\mathcal{T},\mathsf{idx}}$ denote the table commitment in $\mathsf{pp}_{\mathsf{idx}}$, and for each $u \in [\bar{N}]$, let $c_{Q,\mathsf{idx},u}$ denote its cached-quotient commitment associated with table position u. Define $c_{\mathcal{T},w_j}$ and $c_{Q,w_j,u}$ analogously. By $\mathbb{F}$-linearity, the corresponding commitments for the folded table are

$$c_{\mathcal{T}_\rho} \coloneqq c_{\mathcal{T},\mathsf{idx}} + \sum_{j\in[d]} \rho^{j+1} c_{\mathcal{T},w_j}, \tag{8}$$

$$c_{Q_\rho,u} \coloneqq c_{Q,\mathsf{idx},u} + \sum_{j\in[d]} \rho^{j+1} c_{Q,w_j,u}. \tag{9}$$

Here $c_{Q_\rho,u}$ denotes the cached-quotient commitment for the folded table at position $u \in [\bar{N}]$. The prover aggregates cached-quotient commitments only for indices in $\mathsf{supp}(\boldsymbol{m})$, so this online step costs $O(Td)$ group operations and never materializes the folded preprocessing data in full.

Online proving: table side. We prove the scalar inclusion Eq. (7) via the `cq` machinery recapped above. The prover computes the multiplicity vector $\boldsymbol{m}$ and commits to its interpolant $M(X)$. The verifier then derives β via Fiat–Shamir, after which the prover commits to the reciprocal polynomial $A(X)$ and proves Eq. (2) via a pairing check using the folded commitments above. By sparsity of $\boldsymbol{m}$, the commitments to A and Q use only components indexed by $\mathsf{supp}(\boldsymbol{m})$. Once these components have been folded, their assembly costs $O(T)$; including the online folding across d columns, the table-side cost is $O(Td)$ and independent of N and $\bar{N}$.

The table side also outputs the sumcheck claim

$$z \;\coloneqq\; \sum_{u\in[\bar{N}]} f_u \;=\; \sum_{x\in V} A(x). \tag{10}$$

Because V is a multiplicative subgroup and $\deg(A) < \bar{N}$, we have $\sum_{x\in V} A(x) = \bar{N} \cdot A(0)$ [8]. The verifier therefore obtains z from a single opening of A at $X = 0$ and sets $z \leftarrow \bar{N} \cdot A(0)$. The prover assembles this opening in $O(T)$ group operations: by the sparsity of A's evaluations over V, the KZG quotient at $X = 0$ is a linear combination of only the preprocessed $\mathsf{Commit}(\Delta L_u)$ for $u \in \mathsf{supp}(\boldsymbol{m})$.

Online proving: query side. Preprocessing-based lookup arguments [24,81] are typically univariate, whereas the downstream GKR proof operates over multilinear extensions. Following [12], we keep the table side univariate to retain preprocessing and prove the query side using multilinear extensions, reusing the multilinear commitment to $\boldsymbol{H}_0$.

Let $k_T \coloneqq \lceil \log T \rceil$, $k_d \coloneqq \lceil \log d \rceil$, $\bar{T} \coloneqq 2^{k_T}$, and $\bar{d} \coloneqq 2^{k_d}$. Zero-pad $\boldsymbol{H}_0$ to shape $\bar{T} \times \bar{d}$, and interpret $c_0 \coloneqq \mathsf{Commit}(\boldsymbol{H}_0)$ as a commitment to its MLE $\widetilde{H}_0 : \mathbb{F}^{k_T+k_d} \to \mathbb{F}$, defined on valid indices by

$$\widetilde{H}_0(\iota_T(i), \iota_d(j)) = \boldsymbol{H}_0[i,j], \qquad (i,j) \in [T] \times [d],$$

where $\iota_T : [\bar{T}] \to \{0,1\}^{k_T}$ and $\iota_d : [\bar{d}] \to \{0,1\}^{k_d}$ are the standard boolean encodings. From the transcript, the verifier also derives the MLE $\widetilde{t} : \mathbb{F}^{k_T} \to \mathbb{F}$,

taking $\widetilde{t}(\iota_T(i)) = t_i$ for $i < T$ and N on padded indices, and defines $\widetilde{w}_\rho : \mathbb{F}^{k_d} \to \mathbb{F}$ by $\widetilde{w}_\rho(\iota_d(j)) = \rho^{j+1}$ for $j < d$ and 0 otherwise. Define the folded query MLE

$$\widetilde{a}_\rho(x) \;\coloneqq\; \widetilde{t}(x) \;+\; \sum_{y \in \{0,1\}^{k_d}} \widetilde{w}_\rho(y) \cdot \widetilde{H}_0(x, y), \quad x \in \mathbb{F}^{k_T}. \tag{11}$$

Then

$$\widetilde{a}_\rho(\iota_T(i)) = \begin{cases} a_i, & i < T, \\ N, & T \le i < \bar{T}. \end{cases}$$

The $\bar{T} - T$ padded positions therefore contribute $\bar{T} - T$ additional occurrences of the dummy table value $b_N = N$, so the padded multiplicity vector satisfies $m_N = \bar{T} - T$. Accordingly, the table-side claim z from Eq. (10) is

$$z = \sum_{i \in [T]} \frac{1}{a_i + \beta} + \frac{\bar{T} - T}{N + \beta}.$$

It remains to establish this same value from the query side. For every $i \in [\bar{T}]$, define

$$u_i \coloneqq \frac{1}{\widetilde{a}_\rho(\iota_T(i)) + \beta},$$

and let $\widetilde{U} : \mathbb{F}^{k_T} \to \mathbb{F}$ be the MLE satisfying $\widetilde{U}(\iota_T(i)) = u_i$; the prover commits to $\widetilde{U}$. We verify the query side via two constraints:

$$z = \sum_{x \in \{0,1\}^{k_T}} \widetilde{U}(x), \qquad \widetilde{U}(x) \cdot \big(\widetilde{a}_\rho(x) + \beta\big) = 1, \quad \forall x \in \{0,1\}^{k_T}.$$

These are combined into a single batched sumcheck: given $\alpha \xleftarrow{\$} \mathbb{F}$ and $\gamma \xleftarrow{\$} \mathbb{F}^{k_T}$, the prover proves

$$\sum_{x \in \{0,1\}^{k_T}} \Big(\alpha \cdot \widetilde{U}(x) + \widetilde{eq}(\gamma, x)\big(\widetilde{U}(x)(\widetilde{a}_\rho(x) + \beta) - 1\big)\Big) = \alpha \cdot z. \tag{12}$$

The sumcheck reduces the claim to evaluating $\widetilde{U}(r)$ and $\widetilde{a}_\rho(r)$ at a random point $r \in \mathbb{F}^{k_T}$. The value $\widetilde{U}(r)$ is obtained by opening the prover's commitment. For the latter, recall that

$$\widetilde{a}_\rho(r) = \widetilde{t}(r) + \sum_{y} \widetilde{w}_\rho(y) \widetilde{H}_0(r, y).$$

The verifier computes $\widetilde{t}(r)$ directly and obtains the inner sum via a new sumcheck that reduces to an opening of $\widetilde{H}_0$ at a random point.

Cost analysis. The one-time preprocessing cost is $O(dN \log N)$ field/group operations via the optimizations of [25,85] and is amortized across inferences for a fixed model. Online, the prover incurs $O(\bar{T}\bar{d})$ work for both the table-side check and the query-side sumcheck. The verifier performs $O(1)$ KZG pairings for the table-side `cq` check and $O(\log T + \log d)$ field operations for the query-side sumchecks.

3.2 zkPosEnc: Efficient Proof for Positional Encoding

Positional encodings inject token-order information into decoder-only Transformers. We consider the two widely used variants: absolute positional embeddings and rotary positional embeddings.

Absolute positional embeddings (APE). For APE-style models (e.g., GPT-2 [61]), the model contains a learned positional matrix $\boldsymbol{P} \in \mathbb{F}^{T_{\max} \times d}$, committed as part of the private weights. Given the token embeddings $\boldsymbol{H}_0 \in \mathbb{F}^{T \times d}$, the positional-augmented activations are

$$\boldsymbol{H}_0^{\mathsf{pe}} \;\coloneqq\; \boldsymbol{H}_0 + \boldsymbol{P}[0{:}T, :].$$

Since this relation is linear, our proof system handles it natively.

Rotary positional embeddings (RoPE). RoPE-style models (e.g., Llama [71] and Qwen [80]) inject position information by rotating the per-head Query/Key vectors inside each attention block. Fix one attention head with head dimension d_h. Let $\boldsymbol{Q}, \boldsymbol{K} \in \mathbb{F}^{T \times d_h}$ denote its query and key matrices after the linear projections, and write $\boldsymbol{q}_m \coloneqq \boldsymbol{Q}[m, :]$ and $\boldsymbol{k}_m \coloneqq \boldsymbol{K}[m, :]$ for their rows at position $m \in [T]$.

RoPE rotates each consecutive coordinate pair of $\boldsymbol{q}_m$ by angle $m\theta_i$, where $\{\theta_i\}_{i \in [d_h/2]}$ are public frequencies fixed by the architecture (e.g., $\theta_i = 10000^{-2i/d_h}$). Concretely,

$$\boldsymbol{q}_m^{\mathsf{rot}} \;=\; \boldsymbol{R}_m \, \boldsymbol{q}_m, \qquad \boldsymbol{R}_m \;=\; \mathrm{diag}\big(\boldsymbol{R}_m^{(0)}, \ldots, \boldsymbol{R}_m^{(d_h/2-1)}\big), \tag{13}$$

where $\mathrm{diag}(\cdot)$ denotes the block-diagonal matrix with the given blocks, and each $2{\times}2$ rotation block is

$$\boldsymbol{R}_m^{(i)} \coloneqq \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}.$$

and $\boldsymbol{k}_m$ is rotated analogously (formal definition in Appendix B). zkLLM [69] treats these rotations as generic $d_h \times d_h$ matrix–vector multiplications and applies a time-optimal matmul protocol [70], incurring $O(Td_h^2)$ prover work per head. While this protocol is optimal for generic dense matrices, it does not exploit the structure of $\boldsymbol{R}_m$: only $2d_h$ of its d_h^2 entries are non-zero, so all but $O(d_h)$ multiplications are against zeros.

We exploit the $2{\times}2$-block-diagonal structure by rewriting each rotation as two Hadamard products. Define the permutation

$$\pi(\boldsymbol{u}) \;\coloneqq\; (-u_1,\, u_0,\, -u_3,\, u_2,\, \ldots,\, -u_{d_h-1},\, u_{d_h-2}),$$

which, on each consecutive pair (u_{2i}, u_{2i+1}), swaps and sign-flips to produce $(-u_{2i+1}, u_{2i})$. Let $\boldsymbol{c}_m, \boldsymbol{s}_m \in \mathbb{F}^{d_h}$ be the cosine/sine vectors that repeat each frequency twice:

$$\begin{aligned} \boldsymbol{c}_m &= (\cos(m\theta_0), \cos(m\theta_0), \ldots, \cos(m\theta_{d_h/2-1}), \cos(m\theta_{d_h/2-1})), \\ \boldsymbol{s}_m &= (\sin(m\theta_0), \sin(m\theta_0), \ldots, \sin(m\theta_{d_h/2-1}), \sin(m\theta_{d_h/2-1})). \end{aligned}$$

A direct expansion of each 2×2 block then collapses Eq. (13) into two Hadamard products:

$$\boldsymbol{q}_m^{\mathsf{rot}} \;=\; \boldsymbol{q}_m \circ \boldsymbol{c}_m \;+\; \pi(\boldsymbol{q}_m) \circ \boldsymbol{s}_m, \quad \boldsymbol{k}_m^{\mathsf{rot}} \;=\; \boldsymbol{k}_m \circ \boldsymbol{c}_m \;+\; \pi(\boldsymbol{k}_m) \circ \boldsymbol{s}_m. \tag{14}$$

Stacking positions gives the batched form

$$\boldsymbol{Q}^{\mathsf{rot}} \;=\; \boldsymbol{Q} \circ \boldsymbol{C} + \pi(\boldsymbol{Q}) \circ \boldsymbol{S}, \qquad \boldsymbol{K}^{\mathsf{rot}} \;=\; \boldsymbol{K} \circ \boldsymbol{C} + \pi(\boldsymbol{K}) \circ \boldsymbol{S},$$

where $\boldsymbol{C}, \boldsymbol{S} \in \mathbb{F}^{T \times d_h}$ stack $\{\boldsymbol{c}_m\}$ and $\{\boldsymbol{s}_m\}$ row-wise, and $\pi(\cdot)$ is applied row-wise. To bind the signed permutation π, we write the terminal sumcheck point as $(\boldsymbol{r}, \rho)$, where ρ selects the even or odd coordinate within each adjacent pair. Given two openings $q_0 = \widetilde{\boldsymbol{Q}}(\boldsymbol{r}, 0)$ and $q_1 = \widetilde{\boldsymbol{Q}}(\boldsymbol{r}, 1)$ of the same commitment to $\boldsymbol{Q}$, the verifier derives

$$\widetilde{\boldsymbol{Q}}(\boldsymbol{r}, \rho) = (1-\rho)q_0 + \rho q_1, \qquad \widetilde{\pi(\boldsymbol{Q})}(\boldsymbol{r}, \rho) = -(1-\rho)q_1 + \rho q_0.$$

Cost analysis. We verify the resulting Hadamard identity using the dedicated quadratic sumcheck rather than compiling it into generic GKR gates. Since $\boldsymbol{C}, \boldsymbol{S}$ depend only on public positions and frequencies, so we treat them as public constants. The resulting proof system therefore incurs $O(Td_h)$ field operations per head, improving upon the $O(Td_h^2)$ cost of the generic matrix–vector formulation.

3.3 zkDecode: Efficient Proof for the Decoding Procedure

Fix one autoregressive step. Let $\boldsymbol{\ell} = (\ell_0, \ldots, \ell_{N-1}) \in \mathbb{F}^N$ be the quantized logits, with each $\ell_i \in [-B, B)$ under the quantization of §4.1 ($B \ll |\mathbb{F}|$), and let $y \in [N]$ denote the output token.

Greedy decoding. For greedy decoding, correctness means $\ell_y \geq \ell_i$ for all $i \in [N]$. Taking $\lambda \coloneqq \ell_y$ and the differences $\delta_i \coloneqq \ell_i - \lambda$, the prover proves

$$\delta_y \;=\; 0, \qquad -\delta_i \;\in\; [0, 2B) \quad \forall i \in [N].$$

The constraint $\delta_y = 0$ fixes $\lambda = \ell_y$, and the range check forces $\ell_i \leq \ell_y$ for all i, i.e., $y \in \arg\max_i \ell_i$.

Top-k decoding. For sampling-based decoding, we separately prove the validity of the candidate set, the correctness of the Softmax computation and subsequent sampling over that set. We present the construction for top-k here. The top-p variant follows the same overall structure but selects candidates according to cumulative probability mass; we defer it to Appendix C.6.

The prover fixes a candidate set $S \subseteq [N]$ with $|S| = k$ and a *separator* $\mu \in \mathbb{F}$ witnessing the top-k property, together with the differences $\delta_i \coloneqq \ell_i - \mu$. Unlike greedy's $\lambda = \ell_y$, the separator μ carries no semantic role: any value satisfying $\ell_i \geq \mu$ for $i \in S$ and $\ell_i \leq \mu$ for $i \notin S$ suffices, and the honest prover can take, e.g., $\mu = \min_{u \in S} \ell_u$. To express "in S" vs "out of S" without explicitly iterating

over $[N] \setminus S$, we use a boolean selector $\boldsymbol{\chi} \in \{0,1\}^N$ with $\chi_i = 1 \iff i \in S$. The structural constraints are

$$\sum_{i\in[N]} \chi_i = k, \quad \chi_i(\chi_i - 1) = 0 \ \ \forall i \in [N],$$

together with the masked range checks

$$\chi_i \cdot \delta_i \in [0, 2B), \quad (1-\chi_i)\cdot(-\delta_i) \in [0, 2B) \quad \forall i \in [N].$$

The two masked checks force $\ell_i \geq \mu$ for $i \in S$ and $\ell_i \leq \mu$ for $i \notin S$, which is exactly the separator property defining μ; equivalently, S is a valid top-k set. Linkage to the sampled token y is deferred to the subsequent sampling proof.

Softmax with fused masking. The standard top-k sampling specification computes the *full* softmax $p_i = e^{z_i}/\sum_{m\in[N]} e^{z_m}$ (with $z_i \coloneqq \ell_i/\tau$ and temperature $\tau > 0$), truncates to S, and renormalizes to $q_i = p_i/\sum_{j\in S} p_j$ for $i \in S$. Substituting and canceling the shared factor $\sum_m e^{z_m}$ in numerator and denominator yields the equivalent *restricted* form

$$q_i \ = \ \frac{e^{z_i}}{\sum_{j\in S} e^{z_j}} \qquad (i \in S). \tag{15}$$

Thus, the full-softmax normalization and the subsequent renormalization over S collapse into a single normalization over S. However, the arithmetic circuit is fixed across proofs and cannot be reshaped per S. It must therefore produce an output at every index $i \in [N]$, with the output at $i \notin S$ forced to 0. We handle this below via a χ-masked expression.

We prove Eq. (15) from the committed $\boldsymbol{\delta}$ and $\boldsymbol{\chi}$ together with a private auxiliary normalization shift. Since $\boldsymbol{\delta} = \boldsymbol{\ell} - \mu$, shift invariance gives

$$q_i = \frac{\exp(\delta_i/\tau)}{\sum_{j\in S}\exp(\delta_j/\tau)}, \qquad i \in S.$$

Following zkLLM [69], we avoid explicitly forming the denominator and proving the resulting division. Instead, the prover supplies an auxiliary shift ζ, whose ideal real value is

$$\zeta \coloneqq \tau \log\left(\sum_{j\in S}\exp(\delta_j/\tau)\right).$$

For this shift,

$$q_i = \exp\left(\frac{\delta_i - \zeta}{\tau}\right), \qquad i \in S.$$

For the quantized proof, define

$$x_i \coloneqq \chi_i(\delta_i - \zeta), \quad \bar{e}_i \coloneqq \left\lfloor \theta \exp\left(\frac{x_i}{\tau}\right)\right\rfloor, \quad p'_i \coloneqq \chi_i \bar{e}_i, \quad \theta = 2^Q.$$

Thus, the inner mask fixes the lookup inputs outside S to zero, while the outer mask forces the corresponding probabilities to zero. The masked relations are

enforced by sumcheck, and the relation between x_i and $\bar{e}_i$ is proved using exp lookups.

As in zkLLM, we do not directly verify the non-arithmetic log-sum-exp expression defining ζ. Instead, letting

$$\Theta \coloneqq \sum_{i \in [N]} p'_i,$$

we verify

$$\theta - \varepsilon_{\mathsf{sm}} \;\le\; \Theta \;\le\; \theta + \varepsilon_{\mathsf{sm}},$$

where, following the error analysis methodology of zkLLM [69], the tolerance $\varepsilon_{\mathsf{sm}}$ accounts for the fixed-point representation of ζ, the exp-lookup approximation, and output rounding.

Verifiable random sampling. We prove that the output token y was correctly sampled from $\boldsymbol{p}'$ via the *inverse CDF* method: given a random coin r, select the token whose cumulative probability interval contains r.

At inference time, the sampler derives the randomness deterministically from the committed prefix as $r = H(\mathsf{tr}_{<t}) \bmod \Theta$, where H is a public hash modeled as a random oracle. Because r is a function of the prefix alone, the prover cannot re-roll it after observing the logit and thus cannot bias the sampled token.

Given the public output token $y \in [N]$ and randomness r, correct sampling under the inverse CDF method requires

$$C_y \;\le\; r \;<\; C_{y+1}, \tag{16}$$

where $C_j \coloneqq \sum_{i=0}^{j-1} p'_i$ is the cumulative sum up to position j under the canonical ordering by token index. For any $y \notin S$ we have $p'_y = 0$ and hence $C_y = C_{y+1}$, so the interval is empty; Eq. (16) therefore implicitly enforces $y \in S$.

Computing C_y and C_{y+1} via sumcheck. Let $n \coloneqq \lceil \log_2 N \rceil$, and let $\widetilde{p'} : \mathbb{F}^n \to \mathbb{F}$ be the MLE of $\boldsymbol{p}'$ zero-padded to length 2^n. For each public $j \in \{y, y+1\}$, define the public prefix indicator

$$s_j(i) \coloneqq \begin{cases} 1, & i < j, \\ 0, & i \ge j, \end{cases} \qquad i \in [2^n],$$

and let $\widetilde{s}_j$ denote its MLE. Then

$$C_j = \sum_{x \in \{0,1\}^n} \widetilde{s}_j(x)\widetilde{p'}(x), \qquad j \in \{y, y+1\}.$$

Although the prefix $\{0, \ldots, j-1\}$ is not itself a Boolean subcube, the sums above range over the full hypercube and encode the prefix through the public indicator $\widetilde{s}_j$.

The two claims are batched using a random challenge $\eta \xleftarrow{\$} \mathbb{F}$:

$$C_y + \eta C_{y+1} = \sum_{x \in \{0,1\}^n} \big(\widetilde{s}_y(x) + \eta \widetilde{s}_{y+1}(x)\big)\widetilde{p'}(x).$$

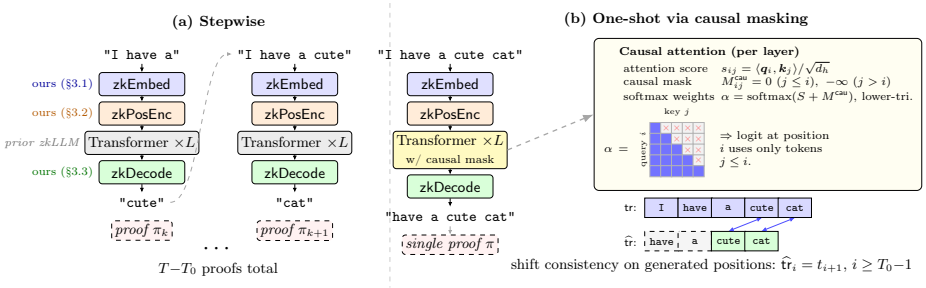


Fig. 2: Stepwise vs. one-shot transcript proving. Stepwise produces one forward-pass proof per generated token; even after recursive aggregation, the prover cost remains super-linear in transcript length since step t conditions on the entire prefix. One-shot proof replaces stepwise proofs for $T - T_0$ tokens with a single causally-masked forward pass.

The verifier evaluates both public prefix-indicator MLEs jointly from the binary representation of y in $O(n)$ time (Appendix C.5). The sumcheck then reduces the two prefix-sum claims to a single opening of $\widetilde{p'}$.

Finally, Eq. (16) is equivalent to the non-negativity of two residuals,

$$r - C_y \in [0, \theta + \varepsilon_{\mathsf{sm}}), \tag{17}$$

$$C_{y+1} - r - 1 \in [0, \theta + \varepsilon_{\mathsf{sm}}). \tag{18}$$

The upper bound follows from $0 \le C_y \le C_{y+1} \le \Theta \le \theta + \varepsilon_{\mathsf{sm}}$.

Cost analysis. Greedy decoding requires (N) logical range checks on logit differences, together with sumcheck proofs that bind these differences to the logits and enforce a zero difference at the selected index. For top-k decoding, the prover work comprises:

- *Candidate set.* $2N$ masked range checks for $\chi_i \delta_i, (1-\chi_i)(-\delta_i) \in [0, 2B)$, and sumchecks for the structural constraints $\sum_i \chi_i = k$ and $\chi_i(\chi_i - 1) = 0$.
- *Softmax.* $O(N)$ exp lookups, $O(N)$-work sumchecks that bind $x_i = \chi_i(\delta_i - \zeta)$ and $p'_i = \chi_i \bar{e}_i$.
- *Sampling.* $O(N)$-work batched sumcheck for C_y and C_{y+1}.

4 zkAgent: One-Shot Verifiable Agent Execution

This section lifts the next-token proof of §3 to *agent executions*: transcripts that interleave model-generated tokens with externally injected tool observations. Whereas prior zkLLM work [69,60] would prove each generation step independently, we prove the entire transcript with a single forward pass, avoiding the per-step proof overhead (Figure 2). Three contributions support this: a *new input-independent quantization scheme* (§4.1), the *causal-mask construction* (§4.2), and *tool binding* for externally injected tokens (§4.3).

4.1 Weight-Dependent Quantization

Our one-shot transcript proof batches all generation step into a single forward pass. For the batched pass to be equivalent to the stepwise autoregressive one it replaces, the quantized arithmetic circuit must produce identical traces under both evaluation modes, imposing two requirements on the quantization scheme:

- *Input-independence*: per-layer output scales $S_{\text{out}}^{(l)}$ must be fixed independently of the input and prefix length, so the batched and stepwise passes produce identical quantized traces.
- *Completeness*: all honest rescaled values stay inside the fixed lookup range, so lookups never reject on honest executions.

Prior zkML quantization schemes fail to satisfy at least one of these properties. *Calibration-based schemes* [77] choose the smallest scale fitting a calibration set; they can violate completeness when benign activations exceed the calibration range, a common phenomenon in LLMs with activation outliers [21]. *Data-dependent dynamic schemes* [44] derive $S_{\text{out}}^{(l)}$ for each inference, avoiding such failures but breaking input-independence. A third option—a purely *dimension-only* bound $d \cdot 2^{2Q}$—satisfies both properties mechanically but assumes every coordinate's input and weight simultaneously saturate at 2^Q, a configuration real LLM weights never approach. On GPT-2 this may collapses perplexity at practical bit-widths (Appendix F).

We introduce *weight-dependent quantization*, the first zkML quantization that attains input-independence and unconditional completeness without severe precision loss.

Weight-dependent worst-case bound. At commitment time, each layer's real weights $W^{(l),\text{real}}$ are quantized to signed integers $W^{(l)}$ with magnitude bounded by 2^Q via per-tensor symmetric quantization, i.e. $|W_{kj}^{(l)}| \le 2^Q$, and a public scale $S_W^{(l)}$ such that $W_{kj}^{(l)} := \left\lfloor W_{kj}^{(l),\text{real}} / S_W^{(l)} \right\rceil$. The real bias $\boldsymbol{b}^{(l),\text{real}}$ is pre-scaled into the accumulator domain of scale $S_a^{(l)} := S_{\text{in}}^{(l)} \cdot S_W^{(l)}$. Let $\boldsymbol{H}^{(l)}$ denote the integer input to layer l with $|\boldsymbol{H}^{(l)}[t,k]| \le 2^Q$. The integer accumulator

$$\boldsymbol{Y}^{(l)}[t,j] \;=\; \textstyle\sum_k \boldsymbol{H}^{(l)}[t,k] \cdot W_{kj}^{(l)} + b_j^{(l)}$$

represents the real value $\boldsymbol{Y}^{(l)}[t,j] \cdot S_a^{(l)}$. The triangle inequality gives

$$|\boldsymbol{Y}^{(l)}[t,j]| \;\le\; 2^Q \cdot \|W^{(l)}[:,j]\|_1 + |b_j^{(l)}|, \quad (t,j) \in [T] \times [d], \tag{19}$$

where $\|W^{(l)}[:,j]\|_1 := \sum_k |W_{kj}^{(l)}|$ is the ℓ_1 norm of column j. We therefore define the input-independent layer bound

$$B_l := \max_j \left(2^Q \|W^{(l)}[:,j]\|_1 + |b_j^{(l)}| \right), \tag{20}$$

which guarantees $|\boldsymbol{Y}^{(l)}[t,j]| \le B_l$ for every input position t and output coordinate j. Since B_l depends only on the committed layer parameters, it is fixed at model-commitment time and reused across all inputs and prefix lengths. To requantize

back to magnitude $\leq 2^Q$ we set

$$S_{\text{out}}^{(l)} \;\coloneqq\; S_a^{(l)} \cdot B_l/2^Q \;=\; S_{\text{in}}^{(l)} \cdot S_W^{(l)} \cdot B_l/2^Q, \tag{21}$$

so the requantization $\boldsymbol{H}^{(l+1)}[t,j] \coloneqq \text{round}\big(2^Q \cdot \boldsymbol{Y}^{(l)}[t,j]/B_l\big)$ yields $|\boldsymbol{H}^{(l+1)}[t,k]| \leq 2^Q$, thereby preserving the magnitude bound $|q| \leq 2^Q$ (full construction in Appendix F).

Input-independence follows because $(B_l, S_a^{(l)}, S_{\text{out}}^{(l)})$ are determined entirely by the committed $(W^{(l)}, \boldsymbol{b}^{(l)}, S_W^{(l)})$; a given prefix is quantized identically in the batched and stepwise executions. *Completeness* follows inductively: given $|\boldsymbol{H}^{(l)}[t,k]| \leq 2^Q$, Eq. (20) forces $|\boldsymbol{Y}^{(l)}| \leq B_l$, and Eq. (21) ensures $|\boldsymbol{H}^{(l+1)}[t,k]| \leq 2^Q$. The base case is enforced at the embedding layer, so the lookup cannot reject on honest execution at any layer.

LayerNorm bound. Fixed-point quantized inference can gradually lose effective precision across depth even though every layer is requantized to the same magnitude bound 2^Q. Each output scale $S_{\text{out}}^{(l)}$ is determined from the input-independent bound B_l; when the real accumulator magnitudes are smaller than B_l, the requantized activations occupy only a fraction of the available integer range. Since these activations become the inputs to subsequent layers, the unused dynamic range can propagate and accumulate across depth, making a fixed Q increasingly coarse. LayerNorm interrupts this propagation by bounding activations in a way that depends only on the layer's parameters and dimension, independent of input magnitude. We give a rigorous derivation of this bound below.

Consider a single row $\boldsymbol{x} \in \mathbb{R}^d$ of the input tensor $\boldsymbol{H} \in \mathbb{R}^{T \times d}$. Let

$$\mu \coloneqq \frac{1}{d}\sum_{k\in[d]} x_k, \qquad \sigma^2 \coloneqq \frac{1}{d}\sum_{k\in[d]} (x_k - \mu)^2,$$

and define the standardized coordinates $\hat{x}_k \coloneqq (x_k - \mu)/\sqrt{\sigma^2 + \varepsilon}$ for a fixed $\varepsilon > 0$. We show that every standardized coordinate has magnitude at most $\sqrt{d-1}$, independently of the input.

(i) *Zero mean and bounded squared norm.* The standardized coordinates sum to zero:

$$\sum_{k\in[d]} \hat{x}_k = \frac{\sum_{k\in[d]} x_k - d\mu}{\sqrt{\sigma^2 + \varepsilon}} = 0.$$

Their squared norm satisfies

$$\sum_{k\in[d]} \hat{x}_k^2 = \frac{\sum_{k\in[d]} (x_k - \mu)^2}{\sigma^2 + \varepsilon} = \frac{d\sigma^2}{\sigma^2 + \varepsilon} \leq d.$$

(ii) *Per-coordinate bound via Cauchy–Schwarz.* Fix any $j \in [d]$. The zero-sum identity gives $\sum_{k\neq j} \hat{x}_k = -\hat{x}_j$. Applying Cauchy–Schwarz to the remaining $d-1$

coordinates yields

$$\begin{aligned}\hat{x}_j^2 = \left(\sum_{k\neq j}\hat{x}_k\right)^2 &\le \left(\sum_{k\neq j}1^2\right)\left(\sum_{k\neq j}\hat{x}_k^2\right)\\ &= (d-1)\left(\sum_{k\in[d]}\hat{x}_k^2 - \hat{x}_j^2\right)\\ &\le (d-1)(d-\hat{x}_j^2).\end{aligned}$$

Rearranging gives

$$d\hat{x}_j^2 \le d(d-1), \qquad \text{hence} \qquad |\hat{x}_j| \le \sqrt{d-1}.$$

The bound holds uniformly over all rows of the standardized tensor.

(iii) *Quantization and affine rescaling.* The bound above determines a fixed normalization scale

$$S_{\text{norm}} \coloneqq \frac{\sqrt{d-1}}{2^Q}.$$

Then we have $q_k \coloneqq \text{round}(\hat{x}_k/S_{\text{norm}}) \le 2^Q$. Thus, standardization provides a fixed magnitude budget without inheriting the input tensor's scale or dynamic range.

The subsequent coordinatewise affine transformation uses the quantized parameters $\gamma_{q,j}^{(l)}$ and $\beta_{q,j}^{(l)}$. Let $S_\gamma^{(l)}$ denote the quantization scale of $\gamma^{(l)}$, and encode the bias at the accumulator scale $S_a^{(l)} \coloneqq S_{\text{norm}}S_\gamma^{(l)}$. The integer multiply–add result is $a_j \coloneqq \gamma_{q,j}^{(l)}q_j + \beta_{q,j}^{(l)}$. Using $|q_j| \le 2^Q$, we obtain

$$\begin{aligned}|a_j| &\le |\gamma_{q,j}^{(l)}|\,|q_j| + |\beta_{q,j}^{(l)}|\\ &\le 2^Q|\gamma_{q,j}^{(l)}| + |\beta_{q,j}^{(l)}|.\end{aligned}$$

We therefore choose the parameter-dependent integer bound

$$B_{\text{aff}}^{(l)} \coloneqq \max_j\left(2^Q|\gamma_{q,j}^{(l)}| + |\beta_{q,j}^{(l)}|\right).$$

Following Eq. (21), we set

$$S_{\text{out}}^{(l)} \coloneqq S_a^{(l)}B_{\text{aff}}^{(l)}/2^Q$$

and requantize the result back to magnitude at most R. Since S_{norm} is fixed by d and R, both $B_{\text{aff}}^{(l)}$ and $S_{\text{out}}^{(l)}$ depend only on the fixed parameters and quantization precision, independently of the input values and prefix length. So the scale of the normalized activations does not inherit the unused range accumulated upstream.

4.2 One-Shot Transcript Proving via Masking

With the quantization scheme of §4.1 in place, we batch *all* model-generated transitions into a *single* proof (Fig. 2). The key idea is a causal attention mask: it forces each position's logit to depend only on preceding tokens, so a single forward pass on the full transcript evaluates all positions in parallel while preserving autoregressive semantics. We formally prove the equivalence in Appendix E.

Setup. Fix a public prompt prefix $\mathsf{tr}_{\mathsf{in}} = (t_0, \ldots, t_{T_0-1})$ and a claimed full transcript $\mathsf{tr} = (t_0, \ldots, t_{T-1}) \in [N]^T$ with $T > T_0$. Let $\mathsf{LLM}_W : [N]^T \to \mathbb{F}^{T\times N}$ output the per-position logits under weights W (§2.5), and $\mathsf{Dec} : \mathbb{F}^N \to [N]$ the agreed decoding rule.

Enforcing autoregressive causality via a causal mask. A naive full-sequence evaluation allows the logits at position i depend on future tokens $t_{>i}$, diverging from sequential generation. In a decoder-only Transformer, however, self-attention is the only sublayer that mixes information across positions—LayerNorm, MLP, and residual connections are position-wise—so enforcing causality within every attention layer suffices. Concretely, for a single head with $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{F}^{T\times d_h}$, we apply the *causal attention mask*

$$\mathsf{Attn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) \;=\; \mathrm{softmax}\!\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top + \boldsymbol{M}^{\mathsf{cau}}}{\sqrt{d_h}}\right)\boldsymbol{V}, \tag{22}$$

where $\boldsymbol{M}^{\mathsf{cau}}_{ij} = 0$ for $j \le i$ and $-\infty$ otherwise. Writing $s_{ij} \coloneqq \langle \boldsymbol{q}_i, \boldsymbol{k}_j\rangle/\sqrt{d_h}$, the resulting softmax weights satisfy

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{k\le i}\exp(s_{ik})}\ (j \le i), \quad \alpha_{ij} = 0\ \text{(otherwise)}, \tag{23}$$

so the logit at position i in the length-T masked run coincides with the sequential-generation logit on prefix $\mathsf{tr}_{\le i}$.

Circuit realization. Since $-\infty$ is not representable in arithmetic circuits, we implement Eq. (23) *by construction*: for each query position i we materialize only the scores $\{s_{ij}\}_{j\le i}$ and set $\alpha_{ij} = 0$ for $j > i$, so masked entries never enter the softmax denominator.

Shift consistency. Then verifying the transcript reduces to a per-position shift-consistency check. Let $L \coloneqq \mathsf{LLM}_W(\mathsf{tr})$ and decoding each logits yields a predicted-token sequence $\widehat{\mathsf{tr}} \in [N]^T$,

$$\widehat{\mathsf{tr}}_i \;\coloneqq\; \mathsf{Dec}\big(L[i,:]\big).$$

Shift consistency (illustrated in Fig. 2(b)) then requires the prediction at position i to match the actual transcript token at position $i{+}1$:

$$\widehat{\mathsf{tr}}_i \;=\; t_{i+1} \quad \text{for all } i \in \{T_0 - 1, \ldots, T - 2\}. \tag{24}$$

4.3 Tool Binding for Verifiable Agent Execution

Tool observations are injected into the transcript and can influence subsequent agent operations, so transcript-level model integrity alone is insufficient: a malicious prover could fabricate tool outputs while still producing a valid transcript [43]. We additionally require *tool integrity and binding*: every injected

observation must be tied to an authenticated execution of the claimed tool on the request parsed from the transcript.

Gated shift consistency. We extend shift consistency to interleaved transcripts via a generation mask $\boldsymbol{g} = (g_0, \ldots, g_{T-2}) \in \{0,1\}^{T-1}$ that identifies which transitions are model-generated:

$$g_i = 1 \iff t_{i+1} \text{ is produced by } \mathsf{LLM}_W \text{ from prefix } \mathsf{tr}_{\le i},$$
$$g_i = 0 \iff t_{i+1} \text{ is injected externally.}$$

With $\widehat{\mathsf{tr}}$ denoting the predicted-token sequence, we enforce

$$g_i\big(\widehat{\mathsf{tr}}_i - t_{i+1}\big) = 0 \qquad \text{for all } i \in \{0, \ldots, T-2\}. \tag{25}$$

leaving injected tokens unconstrained.

A crucial requirement is that $\boldsymbol{g}$ is *derived from the transcript structure*, not chosen by the prover (otherwise the prover could set $\boldsymbol{g} = \mathbf{0}$ and satisfy Eq. (25) vacuously). When the full transcript tr is public, $\boldsymbol{g}$ is uniquely determined by a public, unambiguous transcript grammar. When deployments hide internal reasoning tokens (e.g., chain-of-thought) [15], we distinguish tr from a *public transcript* $\mathsf{tr}^{\mathsf{pub}}$ containing all user/tool tokens and only the user-visible model outputs. Let R denote the hidden span; the provider publishes a binding commitment $c_R := \mathsf{Com}(R)$ and length $L_R := |R|$. Given $(\mathsf{tr}^{\mathsf{pub}}, c_R, L_R)$ and the grammar, the verifier reconstructs the unique segmentation and hence $\boldsymbol{g}$. Any substitution $R \to R'$ either alters a subsequent model output (and is rejected by Eq. (25)) or leaves $\mathsf{tr}^{\mathsf{pub}}$ unchanged (and is therefore harmless to the guarantee).

To rule out *early termination*, we require in the grammar that every model-generated span ends with a designated terminator (e.g., EOS/EOT) that is itself model-generated; premature truncation then either fails parsing or fails Eq. (25).

Tool-call parsing and binding. Each tool step consists of (1) a model-generated *tool-call span* that deterministically parses to a structured request req_i, and (2) a externally injected *tool-observation span*. To ensure that the tool-call schema is correctly followed, we leverage *Coral* [2], a system for verifying a committed stream corresponds to a structured object, to guarantees that each tool-call span conforms to the expected structure and that all necessary parameters are present and valid. We bind tools via *plaintext equality*: the observation span in tr must equal the cleartext token sequence of the authenticated response object resp_i opened from the tool subproof. Thus, subsequent verified generation is guaranteed to condition on the same observation whose provenance and execution are proven. we defer the formal observation-injection attack model and the resulting end-to-end soundness statement to Appendix G.

Consistent with the threat model (§2.6), provider-hosted tools fall into two classes whose integrity reduces to distinct guarantees: networked APIs need *response authenticity under a designated endpoint*, and deterministic sandboxed computations need *execution correctness*. We realize the two classes via zkTLS and zkVM subproofs, respectively, each invoked as a standard protocol with a $(\mathsf{Prove}, \mathsf{Verify})$ interface and bound to tr via plaintext equality.

Table 1: Preprocessing cost of matrix-lookup schemes.

Method	Preprocessing (s)	Size (GiB)
Segment Lookup [18]	8,747.70	6.50
Campanelli et al. [11]	11,296.96	48.00
Ours	**942.43**	**4.52**

Authenticated API tools via zkTLS. For networked tools (e.g., web search and APIs), zkTLS-style systems [79] attest that a recorded TLS session occurred with a specified server. For invocation i with server identity id_i, request req_i, and response resp_i, the prover runs the zkTLS prover to obtain π_i^{io} that is accepted by the zkTLS verifier,

$$\mathsf{Verify}_{\mathsf{io}}(\mathsf{id}_i, \mathsf{req}_i, \mathsf{resp}_i, \pi_i^{\mathsf{io}}) = 1,$$

and binds resp_i into tr.

Deterministic computation tools via Depyler and zkVM. For deterministic sandbox tools, we restrict the model-generated script $\mathsf{program}_i$ to the deterministic Python subset supported by Depyler [57]. Both parties derive the RISC-V program

$$P_i \coloneqq \mathsf{Compile}_{\mathsf{RISC\text{-}V}}\big(\mathsf{Depyler}(\mathsf{program}_i)\big)$$

using the same public, deterministic toolchain. The prover then executes P_i in a zkVM, such as SP1 [68], and supplies a proof π_i^{vm} satisfying

$$\mathsf{Verify}_{\mathsf{vm}}(P_i, \mathsf{in}_i, \mathsf{output}_i, \pi_i^{\mathsf{vm}}) = 1,$$

where in_i denotes any tool input parsed from the transcript. The canonical token encoding of output_i is required to equal the corresponding tool-observation span.

5 Evaluation

We implement zkAgent in C++ and report microbenchmarks for each new subprotocol (zkEmbed, zkPosEnc, and zkDecode), together with performance measurements for one-shot transcript proving and end-to-end agent execution.

5.1 Implementation

Software. Our GKR implementations are adapted from the open-source zkGPT codebase [60]. For polynomial commitments, we use the univariate KZG [37] and the multilinear KZH [36]. We use two interchangeable lookup backends, LogUp [32,58] and Shout [65], offering complementary prover/verifier tradeoffs. Finite-field arithmetic and BN254 elliptic-curve operations use the `mcl` library [49].

Hardware. All experiments are conducted on a server with an Intel Xeon 6982P-C (3.20 GHz, 32 cores) and 512 GB of RAM. We leverage 32-thread parallelization by default.

Table 2: Online cost of matrix-lookup schemes.

Number of queries	2^5	2^6	2^7	2^8	2^9	2^{10}
Prove (s)						
CosetLkup [22]	1602.69	1604.95	1611.45	1617.64	1604.41	1612.59
Segment Lookup [18]	4.98	5.33	6.46	9.75	14.96	25.25
Campanelli et al. [11]	1.12	1.21	1.33	1.63	2.22	3.46
Ours	**0.016**	**0.026**	**0.047**	**0.085**	**0.14**	**0.27**
Verify (s)						
CosetLkup [22]	0.0090	0.0090	0.0090	0.0090	0.0090	0.0090
Segment Lookup [18]	0.03	0.04	0.04	0.03	0.03	0.03
Campanelli et al. [11]	0.0024	0.0025	0.0025	0.0025	0.0025	0.0025
Ours	**0.0064**	**0.0066**	**0.0069**	**0.0072**	**0.0075**	**0.0082**

Quantization. We instantiate the weight-dependent quantization of §4.1 at $Q = 20$ bits, and evaluate the perplexity in Appendix F.

Baselines. We compare against representative prior open-sourced ZK frameworks for LLM inference, including ZKML [13], zkGPT [60], and zkLLM [69]. We do not evaluate the protocols by Lu *et al.* [45] or zkPytorch [77] because neither is open-sourced. We also exclude Hao *et al.* [33]: their released code implements only a subset of nonlinear layers, does not provide an end-to-end implementation.

5.2 Microbenchmarks of Complete-Inference Components

We first microbenchmark the three subprotocols introduced in §3: zkEmbed, zkPosEnc, and zkDecode.

zkEmbed. We evaluate zkEmbed on the GPT-2 embedding table [61] (N=50,257, d=768), varying the number of queried tokens T. We compare against prior matrix lookup protocols, including CosetLkup [22], Segment Lookup [18], and Campanelli *et al.* [11].

Preprocessing costs. Table 1 compares preprocessing time and size across preprocessing-based matrix lookups. zkEmbed achieves the lowest preprocessing time, running 9.3× faster than [18] and 12.0× faster than [11]; it also produces the smallest auxiliary data size. CosetLkup [22] is excluded since it is not preprocessing-based.

Online costs. Table 2 reports the online costs across all matrix lookup scheme. Non-preprocessing baselines are impractical at this scale: CosetLkup [22] takes roughly 1,600 s regardless of T. Among preprocessing based schemes, zkEmbed achieves the lowest proving time: at $T = 2^5$ it takes 0.016 s, 311.3× faster than Segment Lookup (4.98 s) and 70.0× faster than Campanelli *et al.* (1.12 s); at $T = 2^{10}$ it remains 0.27 s while Segment Lookup grows to 25 s. While Campanelli *et al.* achieve a faster verifier ($\sim$ 2.5 ms), zkEmbed's $\sim$ 8 ms is comparable to Segment Lookup and a small fraction of end-to-end zkML verification cost—a worthwhile concession for the prover-side speedup reported above.

Table 3: Preprocessing cost of zkEmbed across models.

Model	Embedding table		Preprocess	
	N	d	time (s)	size (GiB)
GPT-2	50,257	768	942.43	4.52
TinyLlama	32,000	2,048	1,175.30	6.01
GPT-2 Medium	50,257	1,024	1,234.28	6.02
GPT-2 XL	50,257	1,600	1,925.50	9.39
DeepSeek V2	102,400	5,120	12,966.84	60.04
GPT-3	50,257	12,288	14,746.46	72.02
DeepSeek V3	129,280	7,168	18,219.96	84.04

Table 4: Prover and verifier time of RoPE proving.

T	d_h	[70]		Ours	
		Prove (ms)	Verify (ms)	Prove (ms)	Verify (ms)
32	64	35.53	0.04	2.26	0.03
128	64	144.90	0.04	8.90	0.03
512	64	588.47	0.04	35.48	0.04
32	128	122.99	0.04	4.60	0.03
128	128	521.63	0.04	18.15	0.03
512	128	2162.97	0.05	71.82	0.04

zkEmbed for different models. Table 3 reports zkEmbed's preprocessing time and auxiliary storage across representative models. For GPT-2-scale tables (N=50,257, $d \in [768, 1600]$), preprocessing completes in under 33 minutes with 4.5–9.4 GiB of auxiliary data. Scaling to frontier models (GPT-3, DeepSeek-V2/V3) raises the cost to a few hours and tens of GiB, peaking at ~ 5 hours and 84 GiB on DeepSeek-V3. Since preprocessing is a one-time cost amortized across all subsequent inference proofs on a fixed model, these costs remain practical even for the largest models evaluated.

zkPosEnc. Absolute positional embeddings reduce to an offset addition with small overhead, so we focus zkPosEnc's microbenchmarks on rotary positional embeddings (RoPE). We use a single thread here to highlight the algorithmic advantage of zkPosEnc. Table 4 reports RoPE prover and verifier time against the time-optimal matrix multiplication protocol [70], used in zkLLM [69], varying sequence length $T \in \{32, 128, 512\}$ and head dimension $d_h \in \{64, 128\}$. zkPosEnc reduces prover time by 15.7×–30.1× with verifier time essentially unchanged, consistent with the Hadamard form's $O(Td_h)$ cost against the naive $O(Td_h^2)$ arithmetization.

Real-world scale. On Llama-2-7B [71], the model evaluated by zkLLM [69], at T=512 zkPosEnc reduces per-layer RoPE proving from 5.49 s to 0.13 s ($\sim 42\times$), saving ~ 172 s across the full 32 layers.

zkDecode. We microbenchmark zkDecode on the GPT-2 vocabulary (N=50,257) over a T=512-token transcript, reporting amortized per-token prover time, verifier time, and proof size under both decoding rules. For greedy decoding, the cost is dominated by the $O(N)$ range-checked argmax. With the Shout backend, zkDecode amortizes to 0.033 s of prover time, 1.60 ms of verifier time, and a 18.04 KiB proof per generated token; under LogUp, these become 0.041 s, 0.70 ms, and 1.66 KiB. Beyond greedy, top-k further requires candidate-set extraction, softmax normalization over the candidate set, and verifiable random sampling. With Shout, zkDecode amortizes to 0.15 s of prover time, 3.84 ms of verifier time, and a 82.48 KiB proof per token; under LogUp, these become 0.18 s, 2.34 ms, and 7.99 KiB. In an end-to-end proof, zkDecode's range-check lookups are folded into the same batched lookup argument as the inference proof's range checks (§5.3), reusing the padding already paid for there, so its actual marginal cost is smaller than these standalone microbenchmarks suggest.

5.3 One-Shot Transcript Proof

We evaluate one-shot transcript proving on GPT-2, the only model for which prior works provide comparable reference implementations or reported costs. We fix the prompt length to T_0=30 and vary the final transcript length $T \in \{32, 64, 128, 256, 512\}$, so each instance contains $T - T_0$ model-generated tokens. For zkAgent, we report two lookup backends: Shout [65], which yields the fastest prover in our implementation, and LogUp [32,58], which yields smaller proofs and faster verification.

Baseline methodology. To isolate the performance benefit of one-shot proving, we align our proof scope with that of zkGPT. Our measured zkGPT verifier time and proof size exceed those reported in [60] because we include lookup-argument verification, which their released implementation omits. zkLLM does not ship a GPT-2 instantiation, so entries marked † in Table 5 are extrapolated from the per-step A100 costs reported in [60] (15.8 s proving, 0.54 s verification, and 123 KiB proof for a 30-token context). This extrapolation is conservative: in stepwise autoregressive generation, later tokens are proved over longer contexts, so the true per-step cost grows with context length rather than remaining fixed at the 30-token measurement.

Main result. Table 5 reports prover time, verifier time, and proof size. At $T = 512$, zkAgent with the Shout backend proves the entire 482-token continuation in a single proof in 142.45 s, amortizing to 0.30 s per generated token. The resulting proof is 84,589.13 KiB ($\approx$ 82.61 MiB), and verification takes only 2.63 s, since the verifier checks a single transcript-level proof rather than 482 per-token proofs. Against stepwise zkGPT at $T = 512$, this is a 1,047.4× prover speedup (142.45 s vs. 149,192.6 s, about 41 hours) and a 1,657.5× verifier speedup (2.63 s vs. 4,361.09 s). The LogUp backend is slower to prove (200.63 s at $T = 512$) but shrinks the proof to 3,932.09 KiB and verifies in 1.50 s, illustrating the prover/verifier tradeoff between the two backends. For zkLLM, the reported costs are measured on an A100 GPU, while all other systems run on a 32-core CPU server. A careful peak-throughput accounting (Appendix H) gives a $\sim 13\times$ hardware gap in zkLLM's

Table 5: One-shot transcript proving on GPT-2.

T	32	64	128	256	512
Prove (s)					
zkLLM [69]†	31.60	537.20	1,548.40	3,570.80	7,615.60
zkGPT [60]	65.01	1,727.84	7,880.97	33,867.87	149,192.60
zkAgent (LogUp)	**10.18**	**17.79**	**40.70**	**86.67**	**200.63**
zkAgent (Shout)	**8.69**	**14.07**	**29.96**	**65.45**	**142.45**
Verify (s)					
zkLLM [69]†	1.08	18.36	52.92	122.04	260.28
zkGPT [60]	2.96	103.91	457.47	1,358.88	4,361.09
zkAgent (LogUp)	**0.30**	**0.32**	**0.37**	**0.67**	**1.50**
zkAgent (Shout)	**0.55**	**0.66**	**0.85**	**1.26**	**2.63**
Proof size (KiB)					
zkLLM [69]†	246.00	4,182.00	12,054.00	27,798.00	59,286.00
zkGPT [60]	1,279.19	38,298.07	112,719.83	393,307.35	956,043.99
zkAgent (LogUp)	**1,285.63**	**1,668.94**	**2,048.53**	**2,665.16**	**3,932.09**
zkAgent (Shout)	**13,988.44**	**21,653.47**	**32,220.47**	**44,153.19**	**84,589.13**

† Scaled from the per-step zkLLM costs reported in [60] at a 30-token context on an NVIDIA A100 (15.8 s / 0.54 s / 123 KiB); see baseline methodology.

favor. Without normalizing for this gap, zkAgent with Shout is still 53.5× faster to prove at $T = 512$. We omit ZKML from Table 5 as it is prohibitively slow: at the smallest setting ($T = 32$, i.e., $T - T_0 = 2$ generated tokens on a 30-token prompt), its prover already takes 7,148 s, over 3,500 s per generated token; longer contexts are infeasible to measure.

5.4 End-to-End Performance of zkAgent

We evaluate zkAgent end-to-end on two realistic tool-using agent tasks covering the two tool classes of §4.3.

Scope. For each task, we combine the complete one-shot inference proof, including zkEmbed, zkPosEnc, and zkDecode, with tool-specific subproofs under a single statement. Table 6 reports the cost breakdown across the inference proof, the Coral schema proof, and the tool-authenticity proofs.

Weather Agent (zkTLS tool). The agent fetches weather data via a web API and produces a 152-token transcript. Tool integrity is bound by a zkTLS attestation on the HTTP response, and tool-call structure by a Coral schema proof. Under the two inference backends, total proving time is 124.99 s for zkAgent(LogUp) and 95.23 s for zkAgent(Shout), with verification time 0.82 s versus 1.45 s and proof size 3,189.97 KiB versus 47,249.10 KiB.

Coding Assistant (zkVM tool). The agent invokes a deterministic Python sandbox and produces a 143-token transcript. Tool integrity is bound by a zkVM proof on the Rust-transpiled execution, and tool-call structure by a Coral schema proof. Under the two inference backends, total proving time is 122.65 s

Table 6: End-to-end breakdown for agent execution.

Workload	Component	Prove (s)	Verify (s)	Size (KiB)
Weather agent	Inference (LogUp)	117.37	0.78	2,925.53
	Inference (Shout)	87.61	1.41	46,984.66
	Coral schema	2.30	0.033	263.04
	zkTLS tool	5.32	0.011	1.40
	Total (LogUp)	**124.99**	**0.82**	**3,189.97**
	Total (Shout)	**95.23**	**1.45**	**47,249.10**
Coding assistant	Inference (LogUp)	116.47	0.79	2,918.31
	Inference (Shout)	87.00	1.42	46,983.53
	Coral schema	2.29	0.028	263.28
	zkVM tool	3.89	0.18	2,594.69
	Total (LogUp)	**122.65**	**1.00**	**5,776.28**
	Total (Shout)	**93.18**	**1.63**	**49,841.50**

for zkAgent(LogUp) and 93.18 s for zkAgent(Shout), with verification time 1.00 s versus 1.63 s and proof size 5,776.28 KiB versus 49,841.50 KiB.

6 Related Work

ZKPs for ML have progressed from decision trees [83] and CNNs [44,40] to full LLM inference [69,60]. A dominant approach expresses quantized models as arithmetic circuits over finite fields and applies specialized proof systems. zkCNN [44] introduced the GKR-based approach with the advice pattern for non-linear layers; Zen [26] and ZKML [13] provide compiler-oriented frameworks; zkLLM [69] proposed the first ZKP tailored for Transformer inference with dedicated protocols for attention (zkAttn) and non-arithmetic operation (tlookup); and zkGPT [60] further optimized the GKR-based pipeline for LLMs. ZIP [63] demonstrates high-precision (IEEE-754) ZK inference for non-linear activations. NANOZK [74] and Jolt Atlas [10] explore layerwise proofs and lookup-centric approaches, respectively. All of the above focus on proving a single forward pass and do not address the multi-inference, tool-interleaved setting that zkAgent targets. Conversely, these single-forward-pass protocols are orthogonal to and composable with our design: any of them can serve as the Transformer-block backend of our one-shot transcript proof, and our one-shot-with-tool-binding framework immediately lifts single-inference integrity to the multi-inference, tool-interleaved agent setting.

zkPytorch [77] observes that LLM inference can in principle be batched into a single circuit, but its construction neither implements a causal mask to enforce autoregressive dependence, nor proves equivalence to sequential autoregressive generation. Furthermore, its calibration-based quantization lacks completeness on heavy-tailed LLM activations (Appendix F, Table 7). zkPytorch therefore leaves

the problem unresolved; we close both gaps via one-shot transcript proving with formal prefix equivalence under causal masking (§4.2), and weight-dependent quantization that provably keeps every honest activation within the lookup domain (§4.1).

In parallel, VOLE-based interactive proof approaches [75,33,45] show scalable techniques for non-linear functions but impose substantially higher communication and verification overheads than SNARKs. Complementary to inference verification, VeriLoRA [42] provides verifiable LoRA fine-tuning via ZKPs.

Acknowledgments

Claude Code, OpenAI Codex, and Grammarly assisted with drafting and revising manuscript. OpenAI Codex and Claude Code also assisted developing and debugging the research code. The authors independently reviewed all AI-assisted text and code, verified all claims, references, and experimental results, and retain full responsibility for the paper and associated code.

Open Science

We provide the source code of our implementation at `https://github.com/Noi1r/zkAgent-artifacts`

References

1. Abdelnabi, S., Greshake, K., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In: Proceedings of ACM AISec 2023. pp. 79–90. ACM (2023). `https://doi.org/10.1145/3605764.3623985`
2. Angel, S., Celi, S., Margolin, E., Mishra, P., Sander, M., Woods, J.: Coral: Fast succinct non-interactive zero-knowledge CFG proofs. IACR Cryptol. ePrint Arch. **2025**, 1420 (2025), `https://eprint.iacr.org/2025/1420`
3. Anthropic: Claude code. `https://github.com/anthropics/claude-code` (2025), official repository of the API-driven agentic coding tool that runs locally and invokes the Claude API for inference. Accessed 2026-04-14
4. Anthropic: Code execution tool | claude api documentation. Documentation (2026), `https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/code-execution-tool`, server-side sandboxed Python/Bash execution for Claude agents. Accessed 2026-04-09
5. Anthropic: How to implement tool use | claude api docs. Documentation (2026), `https://platform.claude.com/docs/en/agents-and-tools/tool-use/implement-tool-use`, accessed: 2026-02-01
6. Anthropic: Server tools | claude api docs. Documentation (2026), `https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/server-tools`, describes server-side tool execution (web search, code execution) managed by Anthropic. Accessed 2026-04-09

7. Ben-Sasson, E., Chiesa, A., Genkin, D., Tromer, E., Virza, M.: Snarks for C: verifying program executions succinctly and in zero knowledge. In: Advances in Cryptology - CRYPTO 2013. pp. 90–108. Springer (2013). https://doi.org/10.1007/978-3-642-40084-1_6
8. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Ward, N.P.: Aurora: Transparent succinct arguments for R1CS. In: Advances in Cryptology - EUROCRYPT 2019. pp. 103–128. Springer (2019). https://doi.org/10.1007/978-3-030-17653-2_4
9. Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. Algorithmica **79**(4), 1102–1160 (2017). https://doi.org/10.1007/S00453-016-0221-0, https://doi.org/10.1007/s00453-016-0221-0
10. Benno, W., Centelles, A., Douchet, A., Gibran, K.: Jolt atlas: Verifiable inference via lookup arguments in zero knowledge. CoRR **abs/2602.17452** (2026). https://doi.org/10.48550/ARXIV.2602.17452, https://doi.org/10.48550/arXiv.2602.17452
11. Campanelli, M., Faonio, A., Fiore, D., Li, T., Lipmaa, H.: Lookup arguments: Improvements, extensions and applications to zero-knowledge decision trees. In: Public-Key Cryptography - PKC 2024. pp. 337–369. Springer (2024). https://doi.org/10.1007/978-3-031-57722-2_11
12. Campanelli, M., Fiore, D., Gennaro, R.: Natively compatible super-efficient lookup arguments and how to apply them. J. Cryptol. **38**(1), 14 (2025). https://doi.org/10.1007/S00145-024-09535-0, https://doi.org/10.1007/s00145-024-09535-0
13. Chen, B., Waiwitlikhit, S., Stoica, I., Kang, D.: ZKML: an optimizing system for ML inference in zero-knowledge proofs. In: Proceedings of EuroSys 2024. pp. 560–574. ACM (2024). https://doi.org/10.1145/3627703.3650088, code: https://github.com/ddkang/zkml
14. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In: Advances in Cryptology - EUROCRYPT 2023. pp. 499–530. Springer (2023). https://doi.org/10.1007/978-3-031-30617-4_17
15. Chen, X., Sun, Z., Guo, W., Zhang, M., Chen, Y., Sun, Y., Su, H., Pan, Y., Klakow, D., Li, W., Shen, X.: Unveiling the key factors for distilling chain-of-thought reasoning. In: Findings of ACL 2025. pp. 15094–15119. Association for Computational Linguistics (2025), https://aclanthology.org/2025.findings-acl.782/
16. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, P., Ward, N.P.: Marlin: Preprocessing zksnarks with universal and updatable SRS. In: Advances in Cryptology - EUROCRYPT 2020. pp. 738–768. Springer (2020). https://doi.org/10.1007/978-3-030-45721-1_26
17. Chiesa, A., Tromer, E.: Proof-carrying data and hearsay arguments from signature cards. In: Innovations in Computer Science - ICS 2010. pp. 310–331. Tsinghua University Press (2010), http://conference.iiis.tsinghua.edu.cn/ICS2010/content/papers/25.html
18. Choudhuri, A.R., Garg, S., Goel, A., Sekar, S., Sinha, R.: Sublonk: Sublinear prover plonk. Proc. Priv. Enhancing Technol. **2024**(3), 314–335 (2024). https://doi.org/10.56553/POPETS-2024-0080, https://doi.org/10.56553/popets-2024-0080, code: https://github.com/txaty/ark-segmentlookup
19. De Rossi, M., Crapis, D., Ellis, J., Reppel, E.: Erc-8004: Trustless agents. https://eips.ethereum.org/EIPS/eip-8004 (Aug 2025), draft ERC, accessed: 2026-01-14

20. DeepSeek-AI: Deepseek-v3 technical report. CoRR **abs/2412.19437** (2024). https://doi.org/10.48550/ARXIV.2412.19437, https://doi.org/10.48550/arXiv.2412.19437
21. Dettmers, T., Lewis, M., Belkada, Y., Zettlemoyer, L.: Gpt3.int8(): 8-bit matrix multiplication for transformers at scale. In: Advances in Neural Information Processing Systems 35 (2022), http://papers.nips.cc/paper_files/paper/2022/hash/c3ba4962c05c49636d4c6206a97e9c8a-Abstract-Conference.html
22. Di, Z., Xia, L., Nguyen, W.D., Tyagi, N.: Muxproofs: Succinct arguments for machine computation from vector lookups. In: Advances in Cryptology - ASIACRYPT 2024. pp. 236–265. Springer (2024). https://doi.org/10.1007/978-981-96-0935-2_8, code: https://github.com/lucasxia01/mux-proofs-impl
23. Ding, Y., Zhang, L.L., Zhang, C., Xu, Y., Shang, N., Xu, J., Yang, F., Yang, M.: Longrope: Extending LLM context window beyond 2 million tokens. In: Proceedings of ICML 2024. pp. 11091–11104. PMLR / OpenReview.net (2024), https://proceedings.mlr.press/v235/ding24i.html
24. Eagen, L., Fiore, D., Gabizon, A.: cq: Cached quotients for fast lookups. IACR Cryptol. ePrint Arch. **2022**, 1763 (2022), https://eprint.iacr.org/2022/1763
25. Feist, D., Khovratovich, D.: Fast amortized KZG proofs. IACR Cryptol. ePrint Arch. **2023**, 33 (2023), https://eprint.iacr.org/2023/033
26. Feng, B., Qin, L., Zhang, Z., Ding, Y., Chu, S.: ZEN: efficient zero-knowledge proofs for neural networks. IACR Cryptol. ePrint Arch. **2021**, 87 (2021), https://eprint.iacr.org/2021/087
27. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Advances in Cryptology - CRYPTO '86. pp. 186–194. Springer (1986). https://doi.org/10.1007/3-540-47721-7_12
28. Gabizon, A., Williamson, Z.J.: plookup: A simplified polynomial protocol for lookup tables. IACR Cryptol. ePrint Arch. **2020**, 315 (2020), https://eprint.iacr.org/2020/315
29. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. IACR Cryptol. ePrint Arch. **2019**, 953 (2019), https://eprint.iacr.org/2019/953
30. Goldwasser, S., Rothblum, G.N., Kalai, Y.T.: Delegating computation: Interactive proofs for muggles. Electron. Colloquium Comput. Complex. **TR17**, TR17-108 (2017), https://eccc.weizmann.ac.il/report/2017/108
31. Google: Code execution | gemini api. Google AI for Developers Documentation (2026), https://ai.google.dev/gemini-api/docs/code-execution, gemini code execution tool; executes code in Python. Last updated 2026-01-13 (UTC). Accessed 2026-01-15
32. Haböck, U.: Multivariate lookups based on logarithmic derivatives. IACR Cryptol. ePrint Arch. **2022**, 1530 (2022), https://eprint.iacr.org/2022/1530
33. Hao, M., Chen, H., Li, H., Weng, C., Zhang, Y., Yang, H., Zhang, T.: Scalable zero-knowledge proofs for non-linear functions in machine learning. In: Proceedings of USENIX Security 2024. USENIX Association (2024), https://www.usenix.org/conference/usenixsecurity24/presentation/hao-meng-scalable
34. Holtzman, A., Buys, J., Du, L., Forbes, M., Choi, Y.: The curious case of neural text degeneration. In: International Conference on Learning Representations, ICLR 2020. OpenReview.net (2020), https://openreview.net/forum?id=rygGQyrFvH
35. Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A.G., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: Proceedings of CVPR 2018. pp. 2704–2713. Computer

Vision Foundation / IEEE Computer Society (2018). https://doi.org/10.1109/CVPR.2018.00286, http://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
36. Kadianakis, G., Zapico, A., Hafezi, H., Bünz, B.: Kzh-fold: Accountable voting from sublinear accumulation. In: Proceedings of ACM CCS 2025. pp. 409–422. ACM (2025). https://doi.org/10.1145/3719027.3744796
37. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: Advances in Cryptology - ASIACRYPT 2010. pp. 177–194. Springer (2010). https://doi.org/10.1007/978-3-642-17373-8_11
38. Kothapalli, A., Setty, S.T.V.: Hypernova: Recursive arguments for customizable constraint systems. In: Advances in Cryptology - CRYPTO 2024. vol. 14929, pp. 345–379. Springer (2024). https://doi.org/10.1007/978-3-031-68403-6_11
39. Kothapalli, A., Setty, S.T.V., Tzialla, I.: Nova: Recursive zero-knowledge arguments from folding schemes. In: Advances in Cryptology - CRYPTO 2022. pp. 359–388. Springer (2022). https://doi.org/10.1007/978-3-031-15985-5_13
40. Lee, S., Ko, H., Kim, J., Oh, H.: vcnn: Verifiable convolutional neural network based on zk-snarks. IEEE Trans. Dependable Secur. Comput. **21**(4), 4254–4270 (2024). https://doi.org/10.1109/TDSC.2023.3348760
41. Li, X.: A review of prominent paradigms for llm-based agents: Tool use, planning (including rag), and feedback learning. In: Proceedings of COLING 2025. pp. 9760–9779. Association for Computational Linguistics (2025), https://aclanthology.org/2025.coling-main.652/
42. Liao, G., Wang, T., Zhang, S., Zhang, J., Shi, L., Tao, D.: VeriLoRA: Fine-tuning large language models with verifiable security via zero-knowledge proofs. In: Network and Distributed System Security Symposium, NDSS 2026 (2026), https://www.ndss-symposium.org/ndss-paper/verilora-fine-tuning-large-language-models-with-verifiable-security-via-zero-knowledge-proofs/
43. Liu, H., Shou, C., Wen, H., Chen, Y., Fang, R.J., Feng, Y.: Your agent is mine: Measuring malicious intermediary attacks on the LLM supply chain. arXiv:2604.08407 (2026), https://arxiv.org/abs/2604.08407
44. Liu, T., Xie, X., Zhang, Y.: zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In: Proceedings of ACM CCS 2021. pp. 2968–2985. ACM (2021). https://doi.org/10.1145/3460120.3485379
45. Lu, T., Wang, H., Qu, W., Wang, Z., He, J., Tao, T., Chen, W., Zhang, J.: An efficient and extensible zero-knowledge proof framework for neural networks. IACR Cryptol. ePrint Arch. **2024**, 703 (2024), https://eprint.iacr.org/2024/703
46. Lund, C., Fortnow, L., Karloff, H.J., Nisan, N.: Algebraic methods for interactive proof systems. J. ACM **39**(4), 859–868 (1992). https://doi.org/10.1145/146585.146605
47. Merity, S., Xiong, C., Bradbury, J., Socher, R.: Pointer sentinel mixture models. In: International Conference on Learning Representations, ICLR 2017. OpenReview.net (2017), https://openreview.net/forum?id=Byj72udxe
48. Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J.: Distributed representations of words and phrases and their compositionality. In: Advances in Neural Information Processing Systems 26. pp. 3111–3119 (2013), https://proceedings.neurips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html
49. Mitsunari, S.: mcl: A portable pairing-based cryptography library. https://github.com/herumi/mcl
50. NVIDIA Corporation: NVIDIA A100 Tensor Core GPU Architecture: Unprecedented acceleration at every scale. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf (2020)

51. OpenAI: Codex cli. https://github.com/openai/codex (2025), local command-line coding agent powered by the OpenAI Responses API. Accessed 2026-04-14
52. OpenAI: Chatgpt agent. OpenAI Help Center (2026), https://help.openai.com/en/articles/11752874-chatgpt-agent, agent mode overview; lists built-in tools including code interpreter. Accessed 2026-01-15
53. OpenAI: Code interpreter. OpenAI API Documentation (2026), https://platform.openai.com/docs/guides/tools-code-interpreter, defines the Code Interpreter tool for running Python in a sandboxed environment (containers). Accessed 2026-01-15
54. OpenAI: Data analysis with chatgpt. OpenAI Help Center (2026), https://help.openai.com/en/articles/8437071-data-analysis-with-chatgpt, describes ChatGPT's secure code execution environment and passing generated code for execution. Accessed 2026-01-15
55. OpenAI: Function calling | openai api. Documentation (2026), https://platform.openai.com/docs/guides/function-calling, accessed: 2026-02-01
56. OpenAI, Stripe: Agentic commerce protocol. https://developers.openai.com/commerce/guides/get-started (Jan 2026), accessed: 2026-01-14
57. PAIML, contributors: Depyler: Compiles python to rust. GitHub repository (2025), https://github.com/paiml/depyler
58. Papini, S., Haböck, U.: Improving logarithmic derivative lookups using GKR. IACR Cryptol. ePrint Arch. **2023**, 1284 (2023), https://eprint.iacr.org/2023/1284
59. Peng, B., Quesnelle, J., Fan, H., Shippole, E.: Yarn: Efficient context window extension of large language models. In: International Conference on Learning Representations, ICLR 2024. OpenReview.net (2024), https://openreview.net/forum?id=wHBfxhZu1u
60. Qu, W., Sun, Y., Liu, X., Lu, T., Guo, Y., Chen, K., Zhang, J.: zkgpt: An efficient non-interactive zero-knowledge proof framework for LLM inference. In: Proceedings of USENIX Security 2025. pp. 2045–2063. USENIX Association (2025), https://www.usenix.org/conference/usenixsecurity25/presentation/qu-zkgpt, code: https://github.com/security-Anonymous/zkGPT
61. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I.: Language models are unsupervised multitask learners. Technical report, OpenAI (2019), https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
62. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. **21**, 140:1–140:67 (2020), https://jmlr.org/papers/v21/20-074.html
63. Riasi, A., Wang, H., Behnia, R., Vo, V., Hoang, T.: Zero-knowledge AI inference with high precision. In: Proceedings of ACM CCS 2025. pp. 1053–1067. ACM (2025). https://doi.org/10.1145/3719027.3765056
64. Setty, S.T.V.: Spartan: Efficient and general-purpose zksnarks without trusted setup. In: Advances in Cryptology - CRYPTO 2020. pp. 704–737. Springer (2020). https://doi.org/10.1007/978-3-030-56877-1_25
65. Setty, S.T.V., Thaler, J.: Twist and shout: Faster memory checking arguments via one-hot addressing and increments. IACR Cryptol. ePrint Arch. **2025**, 105 (2025), https://eprint.iacr.org/2025/105
66. Setty, S.T.V., Thaler, J., Wahby, R.S.: Unlocking the lookup singularity with lasso. In: Advances in Cryptology - EUROCRYPT 2024. pp. 180–209. Springer (2024). https://doi.org/10.1007/978-3-031-58751-1_7

67. Su, J., Ahmed, M.H.M., Lu, Y., Pan, S., Bo, W., Liu, Y.: Roformer: Enhanced transformer with rotary position embedding. Neurocomputing **568**, 127063 (2024). https://doi.org/10.1016/J.NEUCOM.2023.127063, https://doi.org/10.1016/j.neucom.2023.127063
68. Succinct Labs, contributors: Sp1: A zero-knowledge virtual machine for proving rust/llvm program execution. GitHub repository (2025), https://github.com/succinctlabs/sp1
69. Sun, H., Li, J., Zhang, H.: zkllm: Zero knowledge proofs for large language models. In: Proceedings of ACM CCS 2024. pp. 4405–4419. ACM (2024). https://doi.org/10.1145/3658644.3670334, code: https://github.com/jvhs0706/zkllm-ccs2024
70. Thaler, J.: Time-optimal interactive proofs for circuit evaluation. In: Advances in Cryptology - CRYPTO 2013. pp. 71–89. Springer (2013). https://doi.org/10.1007/978-3-642-40084-1_5
71. Touvron, H., Martin, L., Stone, K., et al.: Llama 2: Open foundation and fine-tuned chat models. CoRR **abs/2307.09288** (2023). https://doi.org/10.48550/ARXIV.2307.09288, https://doi.org/10.48550/arXiv.2307.09288
72. Valiant, P.: Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In: Theory of Cryptography - TCC 2008. pp. 1–18. Springer (2008). https://doi.org/10.1007/978-3-540-78524-8_1
73. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems 30. pp. 5998–6008 (2017), https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
74. Wang, Z.G.: NANOZK: layerwise zero-knowledge proofs for verifiable large language model inference. CoRR **abs/2603.18046** (2026). https://doi.org/10.48550/ARXIV.2603.18046, https://doi.org/10.48550/arXiv.2603.18046
75. Weng, C., Yang, K., Xie, X., Katz, J., Wang, X.: Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. In: Proceedings of USENIX Security 2021. pp. 501–518. USENIX Association (2021), https://www.usenix.org/conference/usenixsecurity21/presentation/weng
76. Wright, A., Andrews, H., Hutton, B., Dennis, G.: JSON schema: A media type for describing JSON documents. IETF Internet-Draft, draft-bhutton-json-schema-01 (2022), https://www.ietf.org/archive/id/draft-bhutton-json-schema-01.html
77. Xie, T., Lu, T., Fang, Z., Wang, S., Zhang, Z., Jia, Y., Song, D., Zhang, J.: zkpytorch: A hierarchical optimized compiler for zero-knowledge machine learning. IACR Cryptol. ePrint Arch. **2025**, 535 (2025), https://eprint.iacr.org/2025/535
78. Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., Song, D.: Libra: Succinct zero-knowledge proofs with optimal prover computation. In: Advances in Cryptology - CRYPTO 2019. pp. 733–764. Springer (2019). https://doi.org/10.1007/978-3-030-26954-8_24
79. Xie, X., Yang, K., Wang, X., Yu, Y.: Lightweight authentication of web data via garble-then-prove. In: Proceedings of USENIX Security 2024. USENIX Association (2024), https://www.usenix.org/conference/usenixsecurity24/presentation/xie-xiang
80. Yang, A., Yang, B., Hui, B., et al.: Qwen2 technical report. CoRR **abs/2407.10671** (2024). https://doi.org/10.48550/ARXIV.2407.10671, https://doi.org/10.48550/arXiv.2407.10671
81. Zapico, A., Buterin, V., Khovratovich, D., Maller, M., Nitulescu, A., Simkin, M.: Caulk: Lookup arguments in sublinear time. In: Proceedings of ACM CCS 2022. pp. 3121–3134. ACM (2022). https://doi.org/10.1145/3548606.3560646

82. Zapico, A., Gabizon, A., Khovratovich, D., Maller, M., Ràfols, C.: Baloo: Nearly optimal lookup arguments. IACR Cryptol. ePrint Arch. **2022**, 1565 (2022), https://eprint.iacr.org/2022/1565
83. Zhang, J., Fang, Z., Zhang, Y., Song, D.: Zero knowledge proofs for decision tree predictions and accuracy. In: Proceedings of ACM CCS 2020. pp. 2039–2053. ACM (2020). https://doi.org/10.1145/3372297.3417278
84. Zhang, J., Liu, T., Wang, W., Zhang, Y., Song, D., Xie, X., Zhang, Y.: Doubly efficient interactive proofs for general arithmetic circuits with linear prover time. In: Proceedings of ACM CCS 2021. pp. 159–177. ACM (2021). https://doi.org/10.1145/3460120.3484767
85. Zhang, T., Ouyang, Y., Zhang, Y.: Dynark: Making groth16 dynamic. IACR Cryptol. ePrint Arch. **2025**, 1897 (2025), https://eprint.iacr.org/2025/1897

A Deployment Variants

This appendix expands on §2.6, which covers two default deployment classes under a single provider-side adversary: fully hosted chat products and API-driven agents. We describe a third class in which the provider-side is split across two independent operators, and show that our construction accommodates it with no protocol changes.

Independent agent-framework operator. In this class, an *agent-framework operator*, distinct from the model provider, runs the agent loop on its own infrastructure and hosts additional tools, while still delegating LLM inference to the model provider's API. SaaS agent platforms and emerging trustless-agent proposals in the Ethereum ecosystem [19] target precisely this pattern.

The operator is a separate principal from the model provider, and the client does not implicitly trust either party. The operator may fabricate tool observations, just as the model provider may fabricate LLM outputs. Our construction of Section 4 accommodates this with no protocol changes. The zkTLS and zkVM primitives of Section 4.3 apply verbatim to operator-hosted tools. The model provider's one-shot inference proof composes into the same bundle unchanged: it yields a claim on the LLM-generated spans of tr, while the operator produces claims on the tool-observation spans it controls. The verifier checks every subproof against c_{tr}, so neither operator can present a consistent bundle without faithfully executing the components it controls.

B Inference Process of Decoder-only LLMs

We briefly review the inference process of decoder-only Large Language Models. Figure 3 gives the full pipeline schematically, annotated with tensor shapes and pointers to the proof protocols introduced in §3.

Step 1: Tokenization. Given a prompt string, a **tokenizer** deterministically converts it into a sequence of token IDs $\mathsf{tr} = (t_0, t_1, \ldots,$
$t_{T-1})$, where $t_i \in \mathcal{V}$ and we view the vocabulary as an index set $\mathcal{V} = [|\mathcal{V}|]$. In most

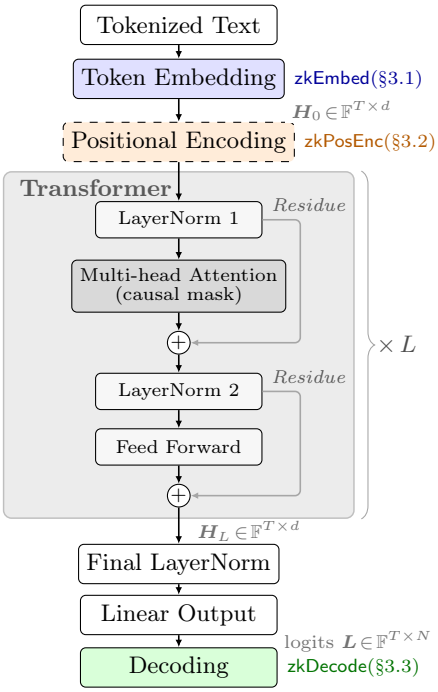


Fig. 3: Detailed inference pipeline of a decoder-only Transformer. The Transformer block (gray) is repeated L times with two residual bypasses. Colored boxes mark stages covered by our new protocols (zkEmbed, zkPosEnc, zkDecode); internals of the Transformer reuse prior zkLLM techniques [69,60].

deployed LLM systems, the tokenizer is treated as public model configuration[1]. Accordingly, we treat the resulting token sequence $\mathsf{tr} \in \mathcal{V}^T$ as part of the public transcript.

Step 2: Token-to-embedding lookup. Let $\boldsymbol{W}_E \in \mathbb{F}^{|\mathcal{V}| \times d}$ be the embedding table. The initial hidden states $\boldsymbol{H}_0 \in \mathbb{F}^{T \times d}$ are obtained by row selection:

$$\boldsymbol{H}_0 \coloneqq \boldsymbol{W}_E(\mathsf{tr}) = (\boldsymbol{W}_{E,t_0,:}, \ldots, \boldsymbol{W}_{E,t_{T-1},:}),$$

i.e., $\boldsymbol{H}_{0,i,:} = \boldsymbol{W}_{E,t_i,:}$ for all $i \in [T]$.

Step 3: Positional encoding. Decoder-only LLMs inject position information using either absolute or rotary positional encodings.

Absolute positional embeddings (APE) If the model uses APE (e.g., GPT-2 [61]), a learnable positional matrix $\boldsymbol{P} \in \mathbb{F}^{T_{\max} \times d}$ is added to embeddings:

$$\boldsymbol{H}_0 \leftarrow \boldsymbol{H}_0 + \boldsymbol{P}[0{:}T, :].$$

[1] LLM manufacturers often release the tokenizer to facilitate token counting and billing for third-party platforms

Rotary positional embeddings (RoPE) For RoPE-based models (e.g., Llama-2 [71], Qwen [80]), position is injected inside attention by rotating the Query and Key vectors. Fix one of the h attention heads, with head dimension $d_h = d/h$. For example, for a query vector at position $m \in [T]$, denoted $\boldsymbol{q}_m \in \mathbb{F}^{d_h}$, the rotated vector is

$$\boldsymbol{q}_m^{\text{rot}} = \boldsymbol{R}_{\Theta,m}^{d_h} \boldsymbol{q}_m,$$

where $\boldsymbol{R}_{\Theta,m}^{d_h}$ is block-diagonal:

$$\boldsymbol{R}_{\Theta,m}^{d_h} = \begin{bmatrix} \boldsymbol{R}_{\Theta,m}^{(0)} & & \\ & \ddots & \\ & & \boldsymbol{R}_{\Theta,m}^{(d_h/2-1)} \end{bmatrix}$$

where each sub-matrix $\boldsymbol{R}_{\Theta,m}^{(i)}$ equals

$$\begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}$$
$$\Theta = \left\{ \theta_i = 10000^{-2i/d_h} \ : \ i \in [d_h/2] \right\}$$

Step 4: Stacked Transformer blocks. The core computation is performed by L Transformer blocks. Let $\boldsymbol{H}_\ell \in \mathbb{F}^{T \times d}$ denote the hidden states after ℓ blocks, with $\boldsymbol{H}_0$ as above. For each block $\ell \in [L]$, we compute

$$\boldsymbol{H}'_{\ell+1} = \boldsymbol{H}_\ell + \text{MSA}_\ell(\text{LN}_\ell(\boldsymbol{H}_\ell)), \tag{26}$$
$$\boldsymbol{H}_{\ell+1} = \boldsymbol{H}'_{\ell+1} + \text{FFN}_\ell(\text{LN}'_\ell(\boldsymbol{H}'_{\ell+1})). \tag{27}$$

Layer normalization (LN) LayerNorm is applied independently to each token position. For a row vector $\boldsymbol{h} \in \mathbb{R}^d$, define

$$\mu(\boldsymbol{h}) \coloneqq \frac{1}{d} \sum_{j \in [d]} h_j, \qquad \sigma^2(\boldsymbol{h}) \coloneqq \frac{1}{d} \sum_{j \in [d]} (h_j - \mu(\boldsymbol{h}))^2,$$
$$\text{LN}(\boldsymbol{h}) \coloneqq \boldsymbol{\gamma} \circ \frac{\boldsymbol{h} - \mu(\boldsymbol{h})\mathbf{1}}{\sqrt{\sigma^2(\boldsymbol{h}) + \varepsilon}} + \boldsymbol{\beta},$$

where $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ are learned affine parameters, $\varepsilon > 0$ is a fixed constant, and $\circ$ denotes the Hadamard product. Applying LN to a matrix $\boldsymbol{H} \in \mathbb{R}^{T \times d}$ is row-wise: $\text{LN}(\boldsymbol{H})[i,:] = \text{LN}(\boldsymbol{H}[i,:])$ for all $i \in [T]$.

Multi-head self-attention (MSA) Let $\boldsymbol{X} = \text{LN}_\ell(\boldsymbol{H}_\ell) \in \mathbb{F}^{T \times d}$. MSA forms queries/keys/values via learned projections:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_{Q,\ell}, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_{K,\ell}, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_{V,\ell}, \tag{28}$$

where $\boldsymbol{W}_{Q,\ell}, \boldsymbol{W}_{K,\ell}, \boldsymbol{W}_{V,\ell} \in \mathbb{F}^{d \times d}$. Split into h heads with head dimension $d_h = d/h$: $\boldsymbol{Q} = [\boldsymbol{Q}^{(0)}; \ldots; \boldsymbol{Q}^{(h-1)}]$ and similarly for $\boldsymbol{K}, \boldsymbol{V}$. For each head $i \in [h]$, scaled

dot-product attention is

$$\boldsymbol{A}^{(i)} = \mathrm{softmax}\left(\frac{\boldsymbol{Q}^{(i)}(\boldsymbol{K}^{(i)})^\top}{\sqrt{d_h}}\right), \tag{29}$$

$$\boldsymbol{Z}^{(i)} = \boldsymbol{A}^{(i)}\boldsymbol{V}^{(i)}, \tag{30}$$

The head outputs are concatenated and projected back:

$$\mathrm{MSA}_\ell(\boldsymbol{X}) = \mathrm{Concat}(\boldsymbol{Z}^{(0)}, \ldots, \boldsymbol{Z}^{(h-1)})\boldsymbol{W}_{O,\ell}, \tag{31}$$

with $\boldsymbol{W}_{O,\ell} \in \mathbb{F}^{d\times d}$.

Feed-forward network (FFN) Each block also contains a position-wise MLP applied independently to each token. A common form is a two-layer MLP with an activation ϕ:

$$\mathrm{FFN}(\boldsymbol{X}) \coloneqq \phi(\boldsymbol{X}\boldsymbol{W}_1 + \mathbf{1}\boldsymbol{b}_1^\top)\boldsymbol{W}_2 + \mathbf{1}\boldsymbol{b}_2^\top,$$

where $\boldsymbol{W}_1 \in \mathbb{R}^{d\times d_{\mathrm{ff}}}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d_{\mathrm{ff}}\times d}$ and $\boldsymbol{b}_1 \in \mathbb{R}^{d_{\mathrm{ff}}}$, $\boldsymbol{b}_2 \in \mathbb{R}^d$. Modern LLMs often use gated variants such as SwiGLU or GeLU.

Step 5: Logits and next-token selection (decoding). After L blocks, the model maps hidden states to vocabulary logits via a linear head. For the last position $T-1$, let $\boldsymbol{H}_{L,T-1,:} \in \mathbb{F}^d$ be the final hidden state vector and $\boldsymbol{W}_{\mathrm{head}} \in \mathbb{F}^{d\times|\mathcal{V}|}$ be the output projection. The logits are

$$\boldsymbol{z}_{T-1} = \mathrm{LN}_{\mathrm{out}}(\boldsymbol{H}_{L,T-1,:}) \cdot \boldsymbol{W}_{\mathrm{head}} \in \mathbb{F}^{|\mathcal{V}|}.$$

In deployed systems, decoding typically converts logits to a categorical distribution using a *temperature* parameter $\tau > 0$:

$$p_\tau(v) \coloneqq \Pr[t_T = v \mid \boldsymbol{z}_{T-1}] = \frac{\exp(\boldsymbol{z}_{T-1}[v]/\tau)}{\sum_{u\in\mathcal{V}}\exp(\boldsymbol{z}_{T-1}[u]/\tau)}, \qquad v \in \mathcal{V}.$$

Smaller τ sharpens the distribution, while larger τ flattens it.

Greedy decoding The greedy rule deterministically selects the most likely token:

$$t_T = \arg\max_{v\in\mathcal{V}} \boldsymbol{z}_{T-1}[v] = \arg\max_{v\in\mathcal{V}} p_\tau(v),$$

which corresponds to taking $\tau \to 0$.

Top-k sampling Let S_k be the set of k tokens with the largest probabilities under p_τ. Define the truncated distribution

$$\tilde{p}_k(v) \coloneqq \begin{cases} \dfrac{p_\tau(v)}{\sum_{u\in S_k} p_\tau(u)} & v \in S_k, \\ 0 & \text{otherwise}, \end{cases}$$

and random sample $t_T \sim \tilde{p}_k(\cdot)$.

Top-p sampling [34] Sort tokens by decreasing probability under p_τ and let S_p be the *smallest* set whose cumulative probability mass exceeds a threshold $p \in (0, 1]$: $\sum_{v \in S_p} p_\tau(v) \geq p$. Then renormalize on S_p (as top-k sampling) and sample t_T from the resulting distribution. Top-p adaptively chooses the candidate set size, unlike fixed-size top-k.

Step 6: Autoregressive loop. Generation is **autoregressive**: given the current context $\mathsf{tr} = (t_0, \ldots, t_{T-1})$, Step 5 yields t_T, which is appended to form a longer context $(t_0, \ldots, t_T)$. The process repeats to generate $t_{T+1}, t_{T+2}, \ldots$ until a stopping condition holds (e.g., an EOS token or a maximum length). Practical implementations may maintain cached state (e.g., KV cache), but the functional semantics remain: each newly generated token is the decoding result computed from the model logits on the preceding context.

C Protocol

C.1 Definitions

We recall the standard definitions of zero-knowledge arguments of knowledge and zero-knowledge polynomial commitments underlying the constructions in the paper. A function $\mu : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is *negligible* if $\mu(\kappa) = \kappa^{-\omega(1)}$; we write $\mathrm{negl}(\kappa)$ for such a function.

Zero-knowledge arguments of knowledge. An argument system for an NP relation $\mathcal{R}$ is a protocol between a PPT prover $\mathcal{P}$ and a PPT verifier $\mathcal{V}$, at the end of which $\mathcal{V}$ is convinced that there exists a witness w with $(x; w) \in \mathcal{R}$ for the common input x. We focus on arguments of *knowledge*: a convincing prover must actually know such a w. Let $\mathcal{G}$ denote the public-parameter generation algorithm, and assume $\mathcal{R}$ is known to both parties.

Definition 1 (Zero-knowledge argument of knowledge). *A tuple $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ is a* zero-knowledge argument of knowledge *for an NP relation $\mathcal{R}$ if the following hold.*

- ***Correctness.*** *For every pp output by $\mathcal{G}(1^\kappa)$ and every $(x, w) \in \mathcal{R}$,*

$$\Pr\big[\, \langle \mathcal{P}(\mathsf{pp}, w), \mathcal{V}(\mathsf{pp}) \rangle(x) = 1 \,\big] = 1.$$

- ***Knowledge soundness.*** *For any PPT prover $\mathcal{P}^*$ there exists a PPT extractor $\mathcal{E}$ such that, given oracle access to the entire execution and random tape of $\mathcal{P}^*$, with $\mathsf{pp} \leftarrow \mathcal{G}(1^\kappa)$, $\pi^* \leftarrow \mathcal{P}^*(x, \mathsf{pp})$ and $w \leftarrow \mathcal{E}^{\mathcal{P}^*}(\mathsf{pp}, x, \pi^*)$,*

$$\Pr\big[\, (x; w) \notin \mathcal{R} \ \wedge\ \mathcal{V}(\pi^*, \mathsf{pp})(x) = 1 \,\big] \leq \mathrm{negl}(\kappa).$$

- ***Zero knowledge.*** *There exists a PPT simulator $\mathcal{S}$ such that for any PPT verifier $\mathcal{V}^*$, any auxiliary input $z \in \{0,1\}^*$, any $(x; w) \in \mathcal{R}$, and pp output by $\mathcal{G}(1^\kappa)$,*

$$\mathsf{View}\big(\langle \mathcal{P}(\mathsf{pp}, w), \mathcal{V}^*(z, \mathsf{pp}) \rangle(x)\big) \ \approx\ \mathcal{S}^{\mathcal{V}^*}(x, z),$$

where $\mathcal{S}$ is given the randomness of $\mathcal{V}^*$ drawn from a polynomial-size space.

We call $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ a *succinct* argument system (equivalently, a zk-SNARK) if the total communication between $\mathcal{P}$ and $\mathcal{V}$ and the verifier's running time are both $\mathrm{poly}(\kappa, |x|, \log |w|)$.

Zero-knowledge polynomial commitment. A zero-knowledge polynomial commitment lets the prover commit to a polynomial f and later reveal $f(t)$ at any queried point t without exposing the rest of f. Let $\mathcal{F}$ be a family of ℓ-variate polynomials over $\mathbb{F}$. A *zero-knowledge polynomial commitment* (zkPC) for $f \in \mathcal{F}$ and $t \in \mathbb{F}^\ell$ consists of three algorithms:

- $\mathsf{pp} \leftarrow \mathsf{zkPC.KeyGen}(1^\kappa)$;
- $\mathsf{com} \leftarrow \mathsf{zkPC.Commit}(f, r_f, \mathsf{pp})$;
- $\big((y, \pi); \{0,1\}\big) \leftarrow \langle \mathsf{zkPC.Open}(f, r_f), \mathsf{zkPC.Verify}(\mathsf{com})\rangle(t, \mathsf{pp})$,

where r_f denotes the prover's commitment randomness.

Definition 2 (zkPC). *A* zkPC *scheme satisfies:*

- ***Completeness.*** *For any $f \in \mathcal{F}$ and $t \in \mathbb{F}^\ell$, with* $\mathsf{pp} \leftarrow \mathsf{zkPC.KeyGen}(1^\kappa)$ *and* $\mathsf{com} \leftarrow \mathsf{zkPC.Commit}(f, r_f, \mathsf{pp})$,

$$\Pr\big[\, \langle \mathsf{zkPC.Open}(f, r_f), \mathsf{zkPC.Verify}(\mathsf{com})\rangle(t, \mathsf{pp}) = 1 \,\big] = 1.$$

- ***Knowledge soundness.*** *For any PPT adversary $\mathcal{A}$ and* $\mathsf{pp} \leftarrow \mathsf{zkPC.KeyGen}(1^\kappa)$, *there exists a PPT extractor $\mathcal{E}$ such that, given any pair* $(\mathsf{pp}, \mathsf{com}^*)$ *and the execution trace of $\mathcal{A}$, $\mathcal{E}$ extracts $f^* \in \mathcal{F}$ and randomness r_{f^*} satisfying*

$$\Pr\left[\begin{array}{l} \big((y^*, \pi^*); 1\big) \leftarrow \langle \mathcal{A}(), \mathsf{zkPC.Verify}(\mathsf{com}^*)\rangle(t, \mathsf{pp}) \\ \wedge\ (f^*, r_{f^*}) \leftarrow \mathcal{E}^{\mathcal{A}}(\mathsf{pp}, \mathsf{com}^*) \\ \wedge\ \mathsf{com}^* = \mathsf{zkPC.Commit}(f^*, r_{f^*}, \mathsf{pp}) \\ \wedge\ f^*(t) \neq y^* \end{array}\right] \leq \mathrm{negl}(\kappa).$$

- ***Zero knowledge.*** *For any security parameter κ, polynomial $f \in \mathcal{F}$,* $\mathsf{pp} \leftarrow \mathsf{zkPC.KeyGen}(1^\kappa)$, *PPT adversary $\mathcal{A}$, and simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$, consider the experiments*

$\mathsf{Real}_{\mathcal{A},f}(\mathsf{pp})$:
(1) $\mathsf{com} \leftarrow \mathsf{zkPC.Commit}(f, r_f)$;
(2) $t \leftarrow \mathcal{A}(\mathsf{com})$;
(3) $(y, \pi) \leftarrow \langle \mathsf{zkPC.Open}(f, r_f), \mathcal{A}\rangle(t)$;
(4) $b \leftarrow \mathcal{A}(\mathsf{com}, y, \pi)$; *output* b.

$\mathsf{Ideal}_{\mathcal{A},\mathcal{S}^f}(\mathsf{pp})$:
(1) $\mathsf{com} \leftarrow \mathcal{S}_1(1^\kappa, \mathsf{pp})$;
(2) $t \leftarrow \mathcal{A}(\mathsf{com})$;
(3) $(y, \pi) \leftarrow \langle \mathcal{S}_2^{f(\cdot)}(\mathsf{com}), \mathcal{A}\rangle(t)$;
(4) $b \leftarrow \mathcal{A}(\mathsf{com}, y, \pi)$; *output* b.

where in the ideal experiment $\mathcal{S}_2$ is given oracle access to f only at the query t. For every PPT $\mathcal{A}$ and every $f \in \mathcal{F}$,

$$\big|\Pr[\mathsf{Real}_{\mathcal{A},f}(\mathsf{pp}) = 1] - \Pr[\mathsf{Ideal}_{\mathcal{A},\mathcal{S}^f}(\mathsf{pp}) = 1]\big| \leq \mathrm{negl}(\kappa).$$

PROTOCOL 1 ([46]) *SumCheck Protocol*

Prover $\mathcal{P}$ claims $v = \sum_{\boldsymbol{x} \in \{0,1\}^n} f(\boldsymbol{x})$ to Verifier $\mathcal{V}$, and $\mathcal{V}$ has the oracle access to f. Let $f_0 = v$.

- In the k-th round where $1 \leq k \leq n-1$:
  - $\mathcal{P}$ sends a univariate polynomial $f_k(X_k) = \sum_{\boldsymbol{x} \in \{0,1\}^{n-k}} f(r_1, \ldots, r_{k-1}, X_k, \boldsymbol{x})$ to $\mathcal{V}$.
  - $\mathcal{V}$ checks $f_{k-1} = f_k(0) + f_k(1)$. If the check passes, $\mathcal{V}$ sends a challenge $r_k \in \mathbb{F}$ to $\mathcal{P}$ and sets $f_k = f_k(r_k)$.
- In the n-th round:
  - $\mathcal{P}$ sends a univariate polynomial $f_n(X_n) = f(r_1, \ldots, r_{n-1}, X_n)$ to $\mathcal{V}$.
  - $\mathcal{V}$ checks $f_{n-1} = f_n(0) + f_n(1)$. If the check passes, $\mathcal{V}$ sends a challenge $r_n \in \mathbb{F}$, and accepts if $f_n(r_n) = f(r_1, \ldots, r_n)$ using one oracle call to f.

Our constructions instantiate the above primitives with concrete building blocks: univariate KZG [37] and multilinear KZH [36] serve as the two polynomial-commitment schemes, while the GKR [30] machinery compiled following the zero-knowledge paradigm of [78] yields the zero-knowledge argument of knowledge used for circuit computations.

C.2 Sumcheck Protocol

The sumcheck protocol [46] enables a verifier $\mathcal{V}$ to verify that the sum of a polynomial $f : \mathbb{F}^n \to \mathbb{F}$ over the boolean hypercube equals a claimed value v, i.e. $v = \sum_{\boldsymbol{x} \in \{0,1\}^n} f(x_0, \ldots, x_{n-1})$, by delegating the heavy computation to a prover $\mathcal{P}$. Directly evaluating the sum requires 2^n evaluations, which is prohibitive for the verifier. In each of the n rounds, the prover sends a univariate polynomial of degree d (the maximum per-variable degree of f), yielding a proof size of $O(dn)$ field elements. The verifier time is $O(dn)$ plus one evaluation of f at a random point. With the dynamic programming technique of [70,78], the prover time is $O(d \cdot 2^n)$. The protocol is complete and sound with soundness error $\epsilon = \frac{dn}{|\mathbb{F}|}$. We provide a detailed description in Protocol 1.

C.3 ML-friendly GKR Protocol

Let C be a depth-d arithmetic circuit arranged into layers indexed by $i \in \{0, \ldots, d\}$, where layer 0 is the output layer and layer d is the input layer. We pad each layer so that it contains 2^{s_i} gates for some $s_i \in \mathbb{N}$, and identify gates in layer i with Boolean labels $z \in \{0,1\}^{s_i}$.

Let $V_i : \{0,1\}^{s_i} \to \mathbb{F}$ map a gate label to its value at layer i, and let $\tilde{V}_i : \mathbb{F}^{s_i} \to \mathbb{F}$ denote the multilinear extension (MLE) of V_i.

Cross-layer value views $V_{i,j}$. For each $i < j$, layer i may use values originating from layer j. We index the subset of layer-j values used by layer i by $\{0,1\}^{s_{i,j}}$ (padding if needed), and define a mapping

$$\pi_{i,j} : \{0,1\}^{s_{i,j}} \to \{0,1\}^{s_j}$$

that specifies which layer-j gate value is referenced. Define

$$V_{i,j}(u) \coloneqq V_j(\pi_{i,j}(u)), \quad u \in \{0,1\}^{s_{i,j}},$$

and let $\tilde{V}_{i,j}$ be the MLE of $V_{i,j}$.

Selection predicates $\widetilde{C}_{i,j}$. Define the Boolean selector

$$C_{i,j} : \{0,1\}^{s_{i,j}} \times \{0,1\}^{s_j} \to \{0,1\}, \quad C_{i,j}(u,x) = \mathbf{1}[x = \pi_{i,j}(u)],$$

and let $\tilde{C}_{i,j} : \mathbb{F}^{s_{i,j}} \times \mathbb{F}^{s_j} \to \mathbb{F}$ denote its MLE. Then for any $r \in \mathbb{F}^{s_{i,j}}$,

$$\tilde{V}_{i,j}(r) = \sum_{x \in \{0,1\}^{s_j}} \tilde{C}_{i,j}(r,x) \cdot \tilde{V}_j(x). \tag{32}$$

This is the identity used to reduce claims about $\tilde{V}_{i,j}$ to claims about $\tilde{V}_j$ via sumcheck.

Unifying dimensions via padding. For a fixed layer $i < d$, we define

$$S_i \coloneqq \max_{j \in \{i+1,\ldots,d\}} s_{i,j},$$

and view all summations in the layer-i check over $\{0,1\}^{S_i}$. For each $j > i$, define the *padding indicator*

$$\widetilde{pad}_{i,j}(y) \coloneqq \prod_{t=s_{i,j}}^{S_i-1} y_t, \quad y \in \{0,1\}^{S_i}.$$

When $\widetilde{pad}_{i,j}(y) = 1$, the suffix $y[s_{i,j} : S_i]$ is forced to all ones, so $y[0 : s_{i,j}]$ effectively ranges over $\{0,1\}^{s_{i,j}}$. This trick lets us sum different-$s_{i,j}$ terms within a single sumcheck instance.

Wiring predicates for layer constraints. We assume (as in [84]) that each gate in layer i has two inputs, where we designate the first input as coming from the "adjacent" view $V_{i,i+1}$, and the second input as coming from $V_{i,j}$ for some $j \in \{i+1,\ldots,d\}$.

For $i < d$ and $j \in \{i+1,\ldots,d\}$, define Boolean wiring predicates over the hypercube:

$$add_{i+1,j} : \{0,1\}^{s_i} \times \{0,1\}^{s_{i+1}} \times \{0,1\}^{s_{i,j}} \to \{0,1\},$$

$$mult_{i+1,j} : \{0,1\}^{s_i} \times \{0,1\}^{s_{i+1}} \times \{0,1\}^{s_{i,j}} \to \{0,1\},$$

where $add_{i+1,j}(z,x,u) = 1$ iff the gate z in layer i is an addition gate whose first input is the x-th entry in $V_{i,i+1}$ and whose second input is the u-th entry in $V_{i,j}$; similarly for $mult_{i+1,j}$. Let

$$\widetilde{add}_{i+1,j}, \widetilde{mult}_{i+1,j}$$

denote the multilinear extensions of $add_{i+1,j}$ and $mult_{i+1,j}$, respectively. These polynomials are fully determined by the circuit description and are efficiently evaluable by $\mathcal{V}$ at arbitrary field points.

Layer relation. For any $g \in \mathbb{F}^{s_i}$ and any $i < d$, we have

$$\begin{aligned}\tilde{V}_i(g) = &\sum_{\substack{x\in\{0,1\}^{s_{i+1}}\\ y\in\{0,1\}^{S_i}}} \sum_{j=i+1}^{d} \widetilde{pad}_{i,j}(y)\cdot \widetilde{add}_{i+1,j}\big(g,x,y[0:s_{i,j}]\big)\cdot \\ &\qquad \big(\tilde{V}_{i,i+1}(x) + \tilde{V}_{i,j}(y[0:s_{i,j}])\big) \\ &+ \sum_{\substack{x\in\{0,1\}^{s_{i+1}}\\ y\in\{0,1\}^{S_i}}} \sum_{j=i+1}^{d} \widetilde{pad}_{i,j}(y)\cdot \widetilde{mult}_{i+1,j}\big(g,x,y[0:s_{i,j}]\big)\cdot \\ &\qquad \tilde{V}_{i,i+1}(x)\cdot \tilde{V}_{i,j}(y[0:s_{i,j}]).\end{aligned} \tag{33}$$

Layer-by-layer reduction. Eq. (33) is checked via sumcheck at each layer, starting from the output layer 0 and proceeding to the input layer d. At layer i, the sumcheck reduces a claim about $\tilde{V}_i(r^{(i)})$ at a random point $r^{(i)} \in \mathbb{F}^{s_i}$ to evaluation claims about the adjacent-layer view $\tilde{V}_{i,i+1}(\cdot)$ and every cross-layer view $\tilde{V}_{i,j}(\cdot)$ referenced by layer i. In a Transformer circuit, cross-layer references arise from two sources: (i) model weights stored in the input layer ($j = d$), and (ii) residual connections that read earlier hidden-state layers (j an intermediate layer). When the reduction reaches a layer j, all evaluation claims accumulated from layers above—whether adjacent-layer or cross-layer—are batched via random linear combination and resolved through a single selector sumcheck (step 3a of Protocol 2), following [84,44]. This layer-by-layer reduction implicitly verifies the correct linkage of every layer's output to the next layer's input: the output V_i *is* the input to layer $i-1$. The formal protocol is presented in Protocol 2.

C.4 Composing GKR with Lookup Arguments

We describe how the GKR-based arithmetic proof and the lookup-based non-arithmetic proof compose into a single layered-circuit proof, expanding on the advice-pattern sketch in the Preliminaries (Section 2.3). We reuse the notation from Section C.3: layers are indexed $i \in \{0,\dots,d\}$ (layer 0 = output, layer d = input), gate labels are $z \in \{0,1\}^{s_i}$, gate values are $V_i(z)$ with MLE $\tilde{V}_i$, and wiring predicates are $\widetilde{add}_{i+1,j}, \widetilde{mult}_{i+1,j}$. For any layer j below i, the portion of V_j read by layer i is denoted $V_{i,j}$ with MLE $\tilde{V}_{i,j}$.

Binding non-arithmetic operations via the input layer. Non-arithmetic operations—such as exponentiation (in Softmax) and range checks (in decoding)—cannot be expressed as arithmetic gates. Following zkCNN [44] and zkGPT [60], we handle them via the *advice* (result-as-witness) pattern: for each non-arithmetic operation $y = f(x)$, both the input x and the output y are placed in the input layer as advice values and committed together with other inputs. Subsequent arithmetic layers read the output y directly from the input layer d via the wiring predicates $\widetilde{add}_{i+1,d}$ and $\widetilde{mult}_{i+1,d}$, so GKR proves the surrounding computation as usual. Consistency between the advice and the actual computation is then enforced by two complementary checks.

1. *Input matching via in-place zero-check.* Using the ML-friendly GKR variant, a gate at layer i can read directly from layer d. To verify that a lookup's input matches the circuit's actual intermediate value, the circuit includes zero-check gates at layer i that compute the difference $\delta = V_{i+1}(z_1) - V_{i,d}(z_2)$. Here $V_{i+1}(z_1)$ is the upstream intermediate value produced at layer $i{+}1$ by the preceding arithmetic computation, and $V_{i,d}(z_2)$ is the corresponding advice copy stored in the input layer d; the subtraction gate thus reads one operand from the adjacent layer and the other from layer d via the ML-friendly wiring. The advice is consistent if and only if $\delta = 0$.
   However, merely including these zero-check gates does not enforce $\delta = 0$. Because their outputs are sinks, they do not affect the ordinary circuit output: a standard GKR layer check can verify that each residual was computed consistently with its source-layer inputs without requiring the residual to vanish.
   At this point, the selector sumcheck in step 3a of Protocol 2 has fixed a random point $g^{(i)} \in \mathbb{F}^{s_i}$ and an evaluation claim

$$v_i = \widetilde{V}_i(g^{(i)}).$$

   For each gate label $z \in \{0,1\}^{s_i}$, let $\mathsf{Eval}_i(z)$ denote the value prescribed by the public layer-i wiring from its source-layer inputs. Let $\mathcal{Z}_i \subseteq \{0,1\}^{s_i}$ be the public set of zero-check gate labels; in our circuit layout, these gates occupy a contiguous suffix beginning at z_0. For $z \in \mathcal{Z}_i$, $\mathsf{Eval}_i(z)$ is the corresponding residual that should equal zero. Define

$$A_i(X) \coloneqq \sum_{z \notin \mathcal{Z}_i} \widetilde{eq}(X, z)\mathsf{Eval}_i(z), \quad D_i(X) \coloneqq \sum_{z \in \mathcal{Z}_i} \widetilde{eq}(X, z)\mathsf{Eval}_i(z).$$

   The ordinary layer relation requires $v_i = A_i(g^{(i)}) + D_i(g^{(i)})$, whereas input matching additionally requires $D_i(g^{(i)}) = 0$.
   To enforce both conditions within the same layer sumcheck, the verifier samples a fresh challenge $\rho_i \xleftarrow{\$} \mathbb{F}$ after $(g^{(i)}, v_i)$ has been fixed and defines the public gate multiplier

$$m_{\rho_i}(z) \coloneqq \begin{cases} 1, & z \notin \mathcal{Z}_i, \\ \rho_i, & z \in \mathcal{Z}_i. \end{cases}$$

The prover and verifier then check the modified layer identity

$$v_i \stackrel{?}{=} \sum_{z\in\{0,1\}^{s_i}} \widetilde{eq}(g^{(i)}, z) m_{\rho_i}(z) \mathsf{Eval}_i(z) = A_i(g^{(i)}) + \rho_i D_i(g^{(i)}). \tag{34}$$

Thus, no additional sumcheck is introduced, and the modified check reduces to the same adjacent- and cross-layer evaluation claims as the ordinary GKR layer check. Algebraically, Eq. (34) randomly combines the ordinary layer relation $v_i - A_i(g^{(i)}) - D_i(g^{(i)}) = 0$ with the zero-residual condition $D_i(g^{(i)}) = 0$.
Completeness. For an honest execution, every zero-check residual is zero, so $D_i(g^{(i)}) = 0$. Equation (34) then reduces to the ordinary layer relation $v_i = A_i(g^{(i)})$ and holds for every ρ_i.
Soundness. For any fixed committed input and advice assignment containing a nonzero zero-check residual, $D_i(X)$ is a nonzero multilinear polynomial of total degree at most s_i. Hence,

$$\Pr[D_i(g^{(i)}) = 0] \le \frac{s_i}{|\mathbb{F}|}.$$

Conditioned on $D_i(g^{(i)}) \neq 0$, all terms in Eq. (34) except the fresh challenge ρ_i are already fixed, so the equation has at most one solution in ρ_i. The zero test therefore adds at most $(s_i + 1)/|\mathbb{F}|$ to the soundness error, in addition to the soundness error of the underlying layer sumcheck.

2. *Non-arithmetic verification via lookup.* A separate lookup argument proves that the advice pairs (x, y) in layer d satisfy the required non-arithmetic relation. For range checks, this means the advice value lies in $[0, B)$; for exponentiation, the pair (x, y) appears in the precomputed exp table.

C.5 Efficient Evaluation of Prefix-Indicator MLEs

We show how the verifier evaluates the public prefix-indicator MLEs used in §3.3 without materializing their length-2^n evaluation vectors. Recall that

$$s_j(a) := \begin{cases} 1, & a < j, \\ 0, & a \ge j, \end{cases} \qquad a \in [2^n].$$

Write the public index j from the most to the least significant bit as

$$j = (b_0, \ldots, b_{n-1}), \qquad b_k \in \{0, 1\},$$

and let $r = (r_0, \ldots, r_{n-1}) \in \mathbb{F}^n$ be the sumcheck point.

Each 1-bit $b_k = 1$ identifies one Boolean subcube contained in the prefix $[0, j)$: the bits preceding k equal $(b_0, \ldots, b_{k-1})$, the k-th bit is 0, and all subsequent bits are free. These subcubes are disjoint and cover $[0, j)$. Summing the MLE basis polynomials over the free suffix gives

$$\widetilde{s}_j(r) = \sum_{k:b_k=1} (1 - r_k) \prod_{\ell<k} \big(b_\ell r_\ell + (1 - b_\ell)(1 - r_\ell)\big). \tag{34}$$

The verifier evaluates Eq. (34) in one left-to-right pass. It initializes $P \leftarrow 1$ and $S \leftarrow 0$. At bit k, if $b_k = 0$, it updates

$$P \leftarrow P(1 - r_k).$$

If $b_k = 1$, it computes $T \leftarrow Pr_k$ and updates

$$S \leftarrow S + (P - T), \qquad P \leftarrow T.$$

At termination,

$$S = \widetilde{s}_j(r), \qquad P = \widetilde{eq}(r, j).$$

Since $s_{j+1} = s_j + \mathbf{1}_{\{j\}}$ on the Boolean hypercube, linearity of multilinear extension gives

$$\widetilde{s}_{j+1}(r) = \widetilde{s}_j(r) + \widetilde{eq}(r, j) = S + P.$$

The scan uses one field multiplication per bit and therefore evaluates both $\widetilde{s}_j(r)$ and $\widetilde{s}_{j+1}(r)$ in $O(n) = O(\log N)$ field operations.

C.6 Top-p Decoding Protocol

Top-p sampling [34] selects the smallest set $S \subseteq [N]$ whose Softmax mass exceeds a public threshold p. The protocol mirrors top-k (§3.3), with two differences: (a) the cutoff is defined by cumulative probability mass under the full-vocabulary Softmax distribution, so S must be verified against that distribution before truncation and renormalization; (b) the size $|S|$ is not fixed by a public k, so the structural constraint $\sum \chi = k$ is replaced by a pair of mass constraints that pin $|S|$ implicitly.

Setup. After the (full) Softmax is computed and committed (denote the integer-scaled probabilities $(p_i)_i$ with public scale), the prover supplies a boolean selector $\boldsymbol{\chi} \in \{0, 1\}^N$ for S and a separator μ playing the role of the probability cutoff.

Constraints. The prover proves

$$\chi_i(\chi_i - 1) = 0 \quad \forall i \in [N],$$

together with the masked range checks

$$\chi_i \cdot (p_i - \mu) \in [0, \theta], \quad (1 - \chi_i) \cdot (\mu - p_i) \in [0, \theta] \quad \forall i \in [N],$$

and the two mass constraints

$$\sum_i \chi_i \cdot p_i \;\geq\; p \cdot \theta, \qquad \Big(\sum_i \chi_i \cdot p_i\Big) - \mu <\; p \cdot \theta.$$

Soundness. As in top-k, the masked range checks force $p_i \geq \mu$ for $i \in S$ and $p_i \leq \mu$ for $i \notin S$, hence $\mu \leq \min_{u \in S} p_u$. The first mass constraint asserts S covers mass at least $p\theta$ (*coverage*); the second, combined with $\mu \leq \min_{u \in S} p_u$, implies $\sum_{u \in S} p_u - \min_{u \in S} p_u < p\theta$ (*minimality*: removing the smallest selected probability drops the mass below $p\theta$). Together these pin $|S|$ implicitly without an explicit cardinality constraint. No pinning of μ is required (the honest prover takes $\mu = \min_{u \in S} p_u$); the link to the sampled token y is established transitively by the sampling proof.

D Preprocessing Lookup

This appendix reviews the cached-quotients (cq) *preprocessed* lookup argument of Eagen et al. [24], which we use as the table-dependent preprocessing primitive throughout the paper. The construction combines Haböck's logarithmic-derivative reduction [32] with univariate KZG polynomial commitments [37], and a one-time preprocessing phase amortizes the *table-side* quotient check across all online proofs.

Notation. We reuse the notation from Section 2.2. Throughout this appendix, let N denote the possibly padded table-domain size and ℓ the possibly padded number of queries, both powers of two. Section 3.1 uses only the table-side preprocessing described here, with the appendix's N corresponding to $\bar{N}$; it replaces the univariate query side over H with a validity-masked multilinear argument. Let $V, H \subset \mathbb{F}$ be multiplicative subgroups with $|V| = N$ and $|H| = \ell$, generated by ω_V and ω_H:

$$V = \{\omega_V^j\}_{j\in[N]}, \quad H = \{\omega_H^i\}_{i\in[\ell]}.$$

Since every element of a multiplicative subgroup of order n satisfies $x^n = 1$, the vanishing polynomials are

$$Z_V(X) = \prod_{x\in V}(X - x) = X^N - 1, \quad Z_H(X) = \prod_{x\in H}(X - x) = X^\ell - 1.$$

Statement. Given a table vector $\boldsymbol{b} = (b_j)_{j\in[N]} \in \mathbb{F}^N$ and a query vector $\boldsymbol{a} = (a_i)_{i\in[\ell]} \in \mathbb{F}^\ell$, the prover proves multiset containment

$$\{a_i\}_{i\in[\ell]} \subseteq \{b_j\}_{j\in[N]}.$$

Equivalently, there exists a matching assignment $\sigma : [\ell] \to [N]$ satisfying $a_i = b_{\sigma(i)}$. Define $m_j := |\sigma^{-1}(j)|$; then $\boldsymbol{m} = (m_j)_{j\in[N]} \in \mathbb{F}^N$ is a multiplicity witness, and the two sides share the same product polynomial:

$$\prod_{i\in[\ell]}(X + a_i) \;=\; \prod_{j\in[N]}(X + b_j)^{m_j}. \tag{35}$$

Applying the formal logarithmic derivative $f \mapsto f'/f$ to both sides converts this product identity into the additive form used by Haböck [32]:

$$\sum_{i\in[\ell]}\frac{1}{a_i + X} \;=\; \sum_{j\in[N]}\frac{m_j}{b_j + X}, \tag{36}$$

which is the identity verified by the cq protocol.

Polynomial encoding and commitments. Let $\mathcal{T}(X) \in \mathbb{F}_{<N}[X]$ be the unique polynomial interpolating the table $\boldsymbol{b}$ on V, and $F(X) \in \mathbb{F}_{<\ell}[X]$ the unique polynomial interpolating the witness $\boldsymbol{a}$ on H:

$$\mathcal{T}(\omega_V^j) = b_j \;\; (j \in [N]), \quad F(\omega_H^i) = a_i \;\; (i \in [\ell]).$$

Throughout, $\mathsf{Commit}(\cdot)$ denotes a univariate KZG commitment [37] with the standard opening interface; `cq` rely on its $\mathbb{F}$-linearity.

A subgroup-sum identity. We use the following univariate sumcheck identity from Aurora [8]: for a multiplicative subgroup $S \subset \mathbb{F}^*$ and any polynomial $p \in \mathbb{F}_{<|S|}[X]$,

$$\sum_{x \in S} p(x) \;=\; |S| \cdot p(0). \tag{37}$$

This follows from $\sum_{x \in S} x^k = 0$ for $1 \le k \le |S| - 1$ and $\sum_{x \in S} 1 = |S|$.

Log-derivative reduction at a random point. The verifier samples $\beta \xleftarrow{\$} \mathbb{F}$. Except with negligible probability over β, it suffices to check Eq. (36) at $X = \beta$. Define pointwise reciprocals on V and H:

$$A(\omega_V^j) \coloneqq \frac{m_j}{\mathcal{T}(\omega_V^j) + \beta} = \frac{m_j}{b_j + \beta},$$
$$B(\omega_H^i) \coloneqq \frac{1}{F(\omega_H^i) + \beta} = \frac{1}{a_i + \beta}.$$

Let $A(X) \in \mathbb{F}_{<N}[X]$ interpolate $\{A(\omega_V^j)\}_{j \in [N]}$ on V, and $B(X) \in \mathbb{F}_{<\ell}[X]$ interpolate $\{B(\omega_H^i)\}_{i \in [\ell]}$ on H. Then Eq. (36) at $X = \beta$ is equivalent to the grand-sum equality

$$\sum_{x \in V} A(x) \;=\; \sum_{x \in H} B(x), \tag{38}$$

which by Eq. (37) reduces to a single scalar constraint:

$$N \cdot A(0) \;=\; \ell \cdot B(0). \tag{39}$$

Well-formedness via divisibility. It remains to enforce that A and B are consistent with $\mathcal{T}, F$ and a multiplicity polynomial. Let $M(X) \in \mathbb{F}_{<N}[X]$ interpolate $\boldsymbol{m}$ on V, i.e., $M(\omega_V^j) = m_j$. The definitions of A and B imply that the following polynomials vanish on V and H, respectively, and hence are divisible by the corresponding vanishing polynomials:

$$A(X) \cdot (\mathcal{T}(X) + \beta) - M(X) = Q_A(X) \cdot Z_V(X), \tag{40}$$
$$B(X) \cdot (F(X) + \beta) - 1 = Q_B(X) \cdot Z_H(X). \tag{41}$$

Given KZG commitments, the verifier can check these divisibility relations succinctly using pairing.

The preprocessing phase. We now specify the offline data produced by the `cq` preprocessing algorithm $\mathsf{Preproc}(\cdot)$ used in the main text. Given a table $\boldsymbol{b} \in \mathbb{F}^N$ with interpolant $\mathcal{T} \in \mathbb{F}_{<N}[X]$ over V, $\mathsf{Preproc}(\boldsymbol{b})$ outputs the following public data for a fixed KZG SRS and domain (V, Z_V):

- Lagrange-basis commitments on V. Commitments to the Lagrange basis polynomials $(L_j(X))_{j \in [N]}$, where $L_j(\omega_V^r) = \mathbf{1}[r = j]$. These allow the prover to commit to any polynomial supported on a sparse set $I \subseteq [N]$ using $O(|I|)$ group operations via linear combination.

– Auxiliary commitments for openings at 0. Commitments to $\Delta L_j(X) := \frac{L_j(X)-L_j(0)}{X}$ for all $j \in [N]$, which are convenient for efficient openings at $X = 0$ when the polynomial is sparse.
– Table commitment. The commitment to the table polynomial $c_{\mathcal{T}} := \mathsf{Commit}(\mathcal{T})$.
– Cached-quotient commitments. Commitments $(c_{Q,j})_{j\in[N]}$ to the cached quotients $Q_j(X)$, each defined by

$$L_j(X) \cdot \mathcal{T}(X) \;=\; b_j \cdot L_j(X) \;+\; Z_V(X) \cdot Q_j(X). \tag{42}$$

Such Q_j exists because $L_j(X)\mathcal{T}(X) - b_j L_j(X)$ vanishes on V (since $L_j(x)\mathcal{T}(x) = b_j L_j(x)$ for all $x \in V$) and is therefore divisible by $Z_V(X)$.

Among the above outputs, the Lagrange-basis and ΔL_j commitments depend only on (N, V) and can be reused across many tables of size N, while $c_{\mathcal{T}}$ and $(c_{Q,j})_{j\in[N]}$ are table-dependent.

Linearity. For a fixed SRS and domain, the table-dependent portion $\mathsf{pp}_{\mathsf{tbl}}(\boldsymbol{b}) := (c_{\mathcal{T}}, (c_{Q,j})_{j\in[N]})$ is $\mathbb{F}$-linear in the table entries:

$$\mathsf{pp}_{\mathsf{tbl}}(\alpha\boldsymbol{b} + \alpha'\boldsymbol{b}') = \alpha\, \mathsf{pp}_{\mathsf{tbl}}(\boldsymbol{b}) + \alpha'\, \mathsf{pp}_{\mathsf{tbl}}(\boldsymbol{b}').$$

The Lagrange-basis and auxiliary commitments depend only on the domain, so they are reused as-is rather than included in this linear combination. This is the property used in the main text to fold the table commitments and cached-quotient commitments of multiple preprocessed instances.

In lookup applications, the multiplicity vector $\boldsymbol{m}$ is sparse: only indices appearing in the witness have $m_j \neq 0$. Let $I := \mathsf{supp}(\boldsymbol{m}) \subseteq [N]$ with $|I| \leq \ell$. Writing A in the Lagrange basis on V,

$$A(X) = \sum_{j\in I} A(\omega_V^j)\, L_j(X),$$

by linearity, the corresponding quotient in Eq. (40) satisfies

$$Q_A(X) = \sum_{j\in I} A(\omega_V^j)\, Q_j(X). \tag{43}$$

Therefore, the prover can form the quotient *commitment* as

$$\mathsf{Commit}(Q_A) = \sum_{j\in I} A(\omega_V^j)\, \mathsf{Commit}(Q_j),$$

using only $O(|I|) = O(\ell)$ group operations, without interpolating or dividing any degree-N polynomials online. The full preprocessing, proving, and verification procedures are given in Protocol 3.

PROTOCOL 3 ([24,25,32]) *Preprocessed Lookup (cached-quotients)*

Let $\mathcal{T} \in \mathbb{F}_{<N}[X]$ interpolate the table $\boldsymbol{b}$ over V and $F \in \mathbb{F}_{<\ell}[X]$ interpolate the witness $\boldsymbol{a}$ over H.

$\mathsf{Preproc}(\boldsymbol{b})$ (offline). Fix a KZG SRS and domain parameters (V, Z_V).

1. Output commitments to the Lagrange basis $(\mathsf{Commit}(L_j))_{j\in[N]}$ and $(\mathsf{Commit}(\Delta L_j))_{j\in[N]}$ for sparse commitments on V.
2. Interpolate $\mathcal{T} \in \mathbb{F}_{<N}[X]$ from $\boldsymbol{b}$ over V, and output the table commitment $c_{\mathcal{T}} \coloneqq \mathsf{Commit}(\mathcal{T})$.
3. For each $j \in [N]$, define Q_j by the cached-quotient identity Eq. (42) and output $c_{Q,j} \coloneqq \mathsf{Commit}(Q_j)$.

Return $\mathsf{pp}(\boldsymbol{b}) \coloneqq \big((\mathsf{Commit}(L_j)), (\mathsf{Commit}(\Delta L_j)), c_{\mathcal{T}}, (c_{Q,j})_{j\in[N]}\big)$.

$\mathsf{Prove}(\mathsf{pp}(\boldsymbol{b}), \boldsymbol{a})$ (online).

1. Compute a multiplicity vector $\boldsymbol{m} = (m_j)_{j\in[N]}$. Let $I \coloneqq \mathsf{supp}(\boldsymbol{m}) = \{j \in [N] : m_j \neq 0\}$, which satisfies $|I| \leq \ell$. Let $M \in \mathbb{F}_{<N}[X]$ interpolate $\boldsymbol{m}$ on V. Using the precomputed Lagrange-basis commitments, compute the sparse commitment
$$c_M \coloneqq \mathsf{Commit}(M) = \sum_{j\in I} m_j \,\mathsf{Commit}(L_j),$$
2. Derive a challenge β through Fiat–Shamir. Define $A \in \mathbb{F}_{<N}[X]$ and $B \in \mathbb{F}_{<\ell}[X]$ by
$$A(\omega_V^j) = \frac{m_j}{\mathcal{T}(\omega_V^j) + \beta}, \qquad B(\omega_H^i) = \frac{1}{F(\omega_H^i) + \beta}.$$
Since $A(\omega_V^j) = 0$ for all $j \notin I$, using the precomputed commitments $(\mathsf{Commit}(L_j))_{j\in[N]}$, the prover computes
$$c_A \coloneqq \mathsf{Commit}(A) = \sum_{j\in I} A(\omega_V^j)\,\mathsf{Commit}(L_j) = \sum_{j\in I} \frac{m_j}{\mathcal{T}(\omega_V^j) + \beta}\,\mathsf{Commit}(L_j),$$
which takes $O(|I|) = O(\ell)$ group operations. $c_B \coloneqq \mathsf{Commit}(B)$ is computed via a standard KZG commitment on H.
3. Provide quotient commitments witnessing Eq. (40)–(41):
   - Table side: form $c_{Q_A} \coloneqq \mathsf{Commit}(Q_A)$ via the cached-quotient combination Eq. (43) using $(c_{Q,j})$.
   - Witness side: compute Q_B on H (e.g., via FFT) and send $c_{Q_B} \coloneqq \mathsf{Commit}(Q_B)$.
4. Open $A(0)$ and $B(0)$ with KZG proofs. For $A(0)$, exploit sparsity using the precomputed $(\mathsf{Commit}(\Delta L_j))$: the KZG opening proof at 0 can be formed as the sparse linear combination
$$\mathsf{Open}_0(A) \;=\; \mathsf{Commit}\left(\frac{A(X) - A(0)}{X}\right) = \sum_{j\in I} A(\omega_V^j)\,\mathsf{Commit}(\Delta L_j),$$
which costs $O(|I|)$ group operations. The opening of $B(0)$ is computed using the standard KZG opening procedure.

Return $\pi = \big(c_M, c_A, c_B, c_{Q_A}, c_{Q_B}, \mathsf{Open}_0(A), \mathsf{Open}_0(B)\big)$

$\mathsf{Verify}(\mathsf{pp}(\boldsymbol{b}), c_F, \pi)$.

1. Recompute β via Fiat–Shamir.
2. Verify the pairing checks for Eq. (40) and Eq. (41).
3. Verify the openings of $A(0)$ and $B(0)$, and check the grand-sum constraint Eq. (39): $N \cdot A(0) = \ell \cdot B(0)$.

Accept iff all checks pass.

Complexity. Let ℓ be the number of queries and N the table size.

- **One-time preprocessing (per fixed table):** $\mathsf{Preproc}(\boldsymbol{b})$ costs $O(N \log N)$ field/group operations, leveraging fast quotient techniques [25,85]. The stored table-dependent data is $O(N)$ group elements.
- **Online proving:** table-side work is dominated by $O(|\,\mathsf{supp}(\boldsymbol{m})|) = O(\ell)$ group operations to form commitments to sparse M, A and the quotient commitment $\mathsf{Commit}(Q_A)$. Query-side work is $O(\ell \log \ell)$ field operations to form Q_B on H, plus $O(\ell)$ commitment/opening work.
- **Verification:** a constant number of pairing checks, plus $O(1)$ field operations for Eq. (39).

Crucially, the *online* complexity is essentially independent of the table size N.

E Prefix Equivalence of Teacher-Forced Causal Forward Pass

This appendix formalizes the correctness of the one-shot proving strategy in Section 4.2. We show that, for a decoder-only Transformer with causal masking, a single teacher-forced forward pass on the full transcript produces intermediate-position logits identical to those obtained by an online autoregressive execution on the corresponding prefix. This equivalence justifies using teacher-forced logits to enforce the shifted next-token constraints in Eq. (25).

Setup. Let $\mathsf{tr} = (t_0, \ldots, t_{T-1}) \in [N]^T$ be a token sequence over a vocabulary of size N. For any $n \in [T]$, consider running the decoder-only Transformer in inference mode on the length-n prefix $\mathsf{tr}_{\le n-1} \coloneqq (t_0, \ldots, t_{n-1})$. This produces a logit matrix $\boldsymbol{L}^{(n)} \in \mathbb{F}^{n \times N}$, whose i-th row is the logit vector at position i, denoted $\boldsymbol{\ell}_i^{(n)} \in \mathbb{F}^N$, i.e., $\boldsymbol{L}^{(n)}[i,:] = \boldsymbol{\ell}_i^{(n)}$.

We rely on the following standard architectural properties of decoder-only Transformers. They are stated explicitly to make the prefix-induction argument self-contained.

1. **Prefix-consistent inputs.** The embedding lookup and all position dependent terms (absolute/rotary encodings or biases) for positions $0, \ldots, n-1$ depend only on their indices, hence are identical whether we run length n or any longer length $n' > n$.

2. **Positionwise non-attention maps.** Every non-attention sublayer (e.g., LayerNorm, MLP, output projections) is applied independently to each position, with residual additions performed entrywise.
3. **Causal masking.** Self-attention is causally masked: position i assigns zero weight to keys $j > i$.

We state the mask in the form implemented in the circuit (Eq. (23) in Section 4.2). Consider a single attention head. Given $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{F}^{n \times d_h}$, define scores $s_{ij} \coloneqq \langle \boldsymbol{q}_i, \boldsymbol{k}_j \rangle / \sqrt{d_h}$ and weights

$$\alpha_{ij} \;\coloneqq\; \begin{cases} \dfrac{\exp(s_{ij})}{\sum_{k \le i} \exp(s_{ik})}, & j \le i, \\ 0, & j > i, \end{cases} \tag{44}$$

and output rows

$$\big(\mathsf{Attn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V})\big)[i, :] \;\coloneqq\; \sum_{j=0}^{n-1} \alpha_{ij}\, \boldsymbol{V}[j, :]. \tag{45}$$

Multi-head attention concatenates heads and applies a positionwise linear map, so it suffices to argue headwise.

Lemma 1 (Prefix invariance of causally masked attention). *Let $m > n$. Let $\boldsymbol{X}^{(m)} \in \mathbb{F}^{m \times d}$ be any input matrix, and let $\boldsymbol{X}^{(n)} \coloneqq \boldsymbol{X}^{(m)}_{[n],:} \in \mathbb{F}^{n \times d}$ be its length-n prefix. Let $\mathsf{Attn}^{(r)}(\cdot)$ denote the masked attention computed on a length-r input using Eqs.* (44) *and* (45)*. Then for every $i \in [n]$,*

$$\big(\mathsf{Attn}^{(m)}(\boldsymbol{X}^{(m)})\big)[i, :] \;=\; \big(\mathsf{Attn}^{(n)}(\boldsymbol{X}^{(n)})\big)[i, :].$$

Proof. Fix any $i \in [n]$. For each $r \in \{n, m\}$, write the per-position projections as

$$\boldsymbol{Q}^{(r)} = \boldsymbol{X}^{(r)} \boldsymbol{W}_Q, \quad \boldsymbol{K}^{(r)} = \boldsymbol{X}^{(r)} \boldsymbol{W}_K, \quad \boldsymbol{V}^{(r)} = \boldsymbol{X}^{(r)} \boldsymbol{W}_V,$$

and denote rows by $\boldsymbol{q}_i^{(r)}, \boldsymbol{k}_j^{(r)}, \boldsymbol{v}_j^{(r)}$. Because $\boldsymbol{X}^{(n)}$ is the prefix of $\boldsymbol{X}^{(m)}$, we have $\boldsymbol{q}_i^{(m)} = \boldsymbol{q}_i^{(n)}$ and $\boldsymbol{k}_j^{(m)} = \boldsymbol{k}_j^{(n)}$, $\boldsymbol{v}_j^{(m)} = \boldsymbol{v}_j^{(n)}$ for all $j \le i$.

Under the causal mask Eq. (44), for both lengths $r \in \{n, m\}$, the attention weights at position i satisfy $\alpha_{ij}^{(r)} = 0$ for all $j > i$, and the normalization is over the identical set $\{0, \ldots, i\}$. Since all scores $s_{ij}^{(r)}$ for $j \le i$ coincide across the two runs, the normalized weights $\{\alpha_{ij}^{(m)}\}_{j \le i}$ and $\{\alpha_{ij}^{(n)}\}_{j \le i}$ match entrywise. Plugging into Eq. (45) gives the claimed equality of output rows.

Theorem 1 (Prefix equivalence of intermediate logits). *For every $i \in [T-1]$, the teacher-forced length-T run and the online prefix run of length $i+1$ produce the same logit vector at position i:*

$$\boldsymbol{\ell}_i^{(T)} \;=\; \boldsymbol{\ell}_i^{(i+1)}.$$

Equivalently, the first $i{+}1$ rows of $\boldsymbol{L}^{(T)}$ equal $\boldsymbol{L}^{(i+1)}$.

Proof. We prove by induction over Transformer layers $\ell = 0, 1, \ldots, L$ that the hidden-state prefixes match. Let $\boldsymbol{H}^{(\ell,n)} \in \mathbb{F}^{n\times d}$ denote the layer-ℓ hidden states in the length-n run.
Base layer. By prefix-consistency of embedding and positional encoding, $\boldsymbol{H}^{(0,T)}_{[i+1],:} = \boldsymbol{H}^{(0,i+1)}$.
Inductive step. Assume $\boldsymbol{H}^{(\ell-1,T)}_{[i+1],:} = \boldsymbol{H}^{(\ell-1,i+1)}$. Every positionwise map (LayerNorm, MLP) preserves row-prefix equality. For the attention sublayer, apply Lemma 1 to conclude that the first $i+1$ rows of the attention output in the length-T run equal those of the length-$(i+1)$ run. Residual additions are entrywise and thus also preserve prefix equality. Therefore $\boldsymbol{H}^{(\ell,T)}_{[i+1],:} = \boldsymbol{H}^{(\ell,i+1)}$.
Logits. The final projection to logits is positionwise linear, so it preserves the same row-prefix identity. In particular, the i-th row matches: $\boldsymbol{\ell}^{(T)}_i = \boldsymbol{\ell}^{(i+1)}_i$.

F Quantization: Details and Accuracy Evaluation

This appendix expands on the weight-dependent quantization scheme of §4.1: it (a) derives the per-layer bounds B_l that determine each output scale $S^{(l)}_{\text{out}}$, (b) details the dyadic requantization protocol and its lookup-table structure, and (c) reports accuracy measurements on GPT-2.
Background. Recall from §2.4 that a real value x is represented by a signed integer q and a scale s, such that $x \approx q \cdot s$. Throughout this appendix, the magnitude bound on any quantized value is $|q| \le 2^Q$ unless stated otherwise. This form is standard in integer-only inference and quantized Transformers [35,60,44]. Multiplying two such integers produces a value with magnitude $\le 2^{2Q}$, and accumulating d such products gives magnitude $\le d \cdot 2^{2Q}$; without intervention, magnitudes would compound across layers. Following zkGPT [60] and zkCNN [44], we insert a *requantization* step after every multiplication–accumulation operation to reset the magnitude back to 2^Q.
Weight-dependent layer bounds. We next derive input-independent magnitude bounds for the different operations in a quantized Transformer. These bounds determine the output scales used by the subsequent requantization steps. Unless stated otherwise, the linear- and attention-layer bounds below are expressed over integer encodings: activations denote requantized integers satisfying $|x_k| \le 2^Q$, weights denote their quantized integer representations, and each bias is encoded in the corresponding accumulator domain with scale $S^{(l)}_a = S^{(l)}_{\text{in}} S^{(l)}_W$. The LayerNorm bound is stated separately in real-value semantics before selecting its integer output scale.

Linear layers (Q/K/V, output projection, FFN up/down). Applying the triangle inequality to $\boldsymbol{Y}^{(l)}[t,j] = \sum_k \boldsymbol{H}[t,k] \cdot W^{(l)}_{kj} + b^{(l)}_j$ entry-wise and using $|\boldsymbol{H}[t,k]| \le 2^Q$ gives Eq. (20), so $B_l = \max_j \left[2^Q \|W^{(l)}[:,j]\|_1 + |b^{(l)}_j|\right]$ uniformly bounds every entry of $\boldsymbol{Y}^{(l)}$ across $(t,j) \in [T] \times [d]$. Since B_l depends only on the quantized weights and biases, it is computed once at model-loading time and reused across all executions.

Attention scores $\boldsymbol{QK}^\top/\sqrt{d_h}$. Since both $\boldsymbol{Q}$ and $\boldsymbol{K}$ are activations, we fall back to the bit-width bound $|s_{ij}| \le \sqrt{d_h} \cdot 2^{2Q}$.

Softmax and attention output $\alpha\boldsymbol{V}$. Following the division-free Softmax protocol of §3.3 and zkLLM [69], each attention row is shifted by its log-sum-exp before the exponentiation lookup. The lookup inputs are therefore non-positive, so under the integer scale $\theta = 2^Q$ its outputs satisfy

$$0 \le \bar{\alpha}_{ij} = \lfloor \theta \exp(x_{ij}) \rceil \le \theta.$$

Moreover, the normalization check ensures $\sum_j \bar{\alpha}_{ij} \le \theta + \varepsilon_{\mathsf{attn}}$. Consequently, for $o_{ik} = \sum_j \bar{\alpha}_{ij} V_{jk}$ and $|V_{jk}| \le 2^Q$,

$$|o_{ik}| \le (\theta + \varepsilon_{\mathsf{attn}}) 2^Q.$$

LayerNorm. LayerNorm is the essential dynamic-range reset that makes weight-dependent bounds practical at depth: because the standardized coordinates satisfy $|\hat{x}_k| \le \sqrt{d-1}$ independently of the input (see §4.1 for the derivation).

Requantization protocol. Given an accumulator a with scale $S_a = S_{\text{in}} \cdot S_W$ and target output scale S_{out} , the requantized integer is

$$q_{\text{out}} \;=\; \text{round}\Big(\frac{C_1^{(l)} \cdot a}{C_2^{(l)}}\Big), \tag{46}$$

in a *dyadic* form [35] chosen that $C_1^{(l)}/C_2^{(l)} \approx S_a/S_{\text{out}}$. The prover provides q_{out} as advice and proves the rounding constraint via a single shifted range check on the signed residual $\delta^{(l)} \coloneqq 2C_1^{(l)} \cdot a - 2C_2^{(l)} q_{\text{out}}$ as in zkGPT [60], namely

$$\delta^{(l)} + C_2^{(l)} \;\in\; [0,\, 2C_2^{(l)}), \tag{47}$$

which is equivalent to $\delta^{(l)} \in [-C_2^{(l)},\, C_2^{(l)})$. After requantization, q_{out} is guaranteed to satisfy $|q_{\text{out}}| \le 2^Q$, resetting the magnitude bound for the next layer. Constraint fusion [60] further merges adjacent rounding constraints; the fused check operates on a wider residual.

Parameter selection: $C_1^{(l)}$ **and** $C_2^{(l)}$. Let $\rho^{(l)} \coloneqq \frac{S_a^{(l)}}{S_{\text{out}}^{(l)}}$. Because $\rho^{(l)}$ may be much smaller than one, we choose the smallest power-of-two denominator that makes its scaled numerator have Q significant bits:

$$n^{(l)} \coloneqq \left\lceil \log_2\Big(1/\rho^{(l)}\Big) \right\rceil, \tag{48}$$

$$C_2^{(l)} \coloneqq 2^{\,n^{(l)}+Q-1}, \tag{49}$$

$$C_1^{(l)} \coloneqq \left\lfloor \rho^{(l)} C_2^{(l)} \right\rfloor. \tag{50}$$

Thus, $\rho^{(l)} C_2^{(l)} \in [2^{Q-1}, 2^Q)$, preventing $C_1^{(l)}$ from vanishing while retaining Q-bit relative precision.

Empirical failure of calibration-based scaling. We evaluate calibration-derived range failures across four LLMs of increasing scale. For each model, we

Table 7: Fraction of test tokens that exceed at least one calibration-derived bound (%). Calibration on 50K WikiText-2 train tokens; evaluation on 50K tokens each from WikiText-2 test, a non-overlapping slice of WikiText-103 test, and C4 validation.

Model	Params	WT2	WT103	C4
GPT-2	0.12B	0.218	0.162	0.438
TinyLlama	1.1B	0.388	0.610	2.512
Llama-2-7B	7B	0.644	0.770	2.588
Llama-2-13B	13B	1.188	1.102	5.006

run calibration on 50K WikiText-2 train tokens and use the maximum absolute floating-point value observed for each monitored tensor as its bound. We monitor the Q/K/V and attention-output projections, attention scores, residual outputs, MLP intermediates, and final logits. We then evaluate three held-out sets of 50K tokens each: WikiText-2 test; WikiText-103 test[2]; and C4 validation [62], a web-crawl corpus outside the wiki calibration distribution. A token is counted if any monitored tensor in its forward pass exceeds its corresponding calibrated bound.

Table 7 reports the resulting per-token calibrated-bound exceedance rate. On both wiki test sets, the rate increases monotonically with model size: from 0.218% to 1.188% on WikiText-2 and from 0.162% to 1.102% on WikiText-103. Distribution shift to C4 raises the rate by 2.0–6.5×, reaching 5.006% (2,503 of 50,000 tokens) on Llama-2-13B. These results show that calibration-derived maxima can underestimate runtime activations, leaving the resulting lookup ranges insufficient for some valid executions.

Perplexity. We report perplexity (PPL) in Table 8 on three standard language modeling benchmarks: WikiText-2 and WikiText-103 [47], and C4 [62]. Perplexity is the exponentiated average negative log-likelihood, and therefore directly measures how much the next-token distribution shifts under quantization. A small PPL increase indicates that quantization introduces only minor distortion to the model's predictive distribution. To evaluate the complete quantization scheme, we compare it against a naive dimension-only baseline that applies generic worst-case bounds and immediate requantization throughout the Transformer. The comparison therefore captures the aggregate benefit of our operator-specific bounds and weight-dependent quantization.

[2] The first 50K tokens of WikiText-103 test are byte-identical to WikiText-2 test in the HuggingFace release, so we take tokens 50,000–100,000 instead, yielding a distinct sample from the same wiki domain.

Table 8: Perplexity (PPL) on GPT-2 under our quantization scheme and an unoptimized dimension-only baseline.

Configuration	Q	**WikiText-2**	**WikiText-103**	**C4**
FP32	—	24.36	24.22	32.85
Naive Dim.	18	3.86×10^{23}	3.71×10^{23}	7.21×10^{22}
Ours (full)	18	**24.42**	**24.28**	**32.87**
Naive Dim.	20	1.12×10^{11}	1.15×10^{11}	1.48×10^{10}
Ours (full)	20	**24.36**	**24.22**	**32.84**
Naive Dim.	22	3897.19	3893.44	3131.06
Ours (full)	22	**24.36**	**24.22**	**32.85**

G Tool Binding Details

This appendix expands on §4.3, covering the Depyler transpilation step, the observation-injection attack that motivates Coral, the end-to-end soundness statement, and our tool taxonomy.

Depyler-based transpilation. Depyler [57] is a public open-source transpiler that compiles annotated Python to Rust while preserving program semantics. Given a model-emitted script $\mathsf{program}_i$, Depyler produces $\mathsf{Rust}(\mathsf{program}_i)$, which is then compiled to RISC-V and proved correct under SP1 [68]; semantic preservation makes a proof of the Rust execution a valid proof of the original Python execution. Both $\mathsf{program}_i$ and $\mathsf{Rust}(\mathsf{program}_i)$ are public artifacts and can be re-run by any third party, so the transpilation step itself need not be proved in-circuit, avoiding substantial prover overhead.

Observation-injection attack and schema integrity. Without proving tool-call well-formedness, a malicious prover could label a model-generated span $\mathsf{tr}_{[a]}$ that does not satisfy the public tool-call schema as an invocation, execute a prover-chosen request, and insert the authenticated response as an external observation $\mathsf{tr}_{[b]}$. The tool subproof and all subsequent model-generation checks could then remain valid even though the prescribed agent parser would not have invoked a tool on $\mathsf{tr}_{[a]}$. We prevent this attack by using Coral [2] to prove that the committed call span admits a valid parse under the public tool-call grammar. The request extracted from this parse is bound to the request authenticated by the corresponding zkTLS or zkVM subproof, while the transcript grammar permits an observation span only immediately after such a valid call. Consequently, malformed or ordinary model-generated text cannot authorize an externally injected observation.

End-to-end soundness statement. Let $\boldsymbol{g}$ be the transcript-derived generation mask defined in §4.3. Given commitments c_W and c_{tr} to the model parameters W and the full token transcript tr, respectively, $\mathsf{zkAgent}$'s composed proof attests to:

1. *Model integrity*: every $g_i{=}1$ position satisfies Eq. (25) under LLM_W and Dec;

2. *Schema integrity*: every tool-call span parses to a structured request req_i under the public grammar;
3. *Tool integrity*: every tool-observation span is the authentic output of executing the claimed tool on req_i, attested by a zkTLS proof π_i^{io} for I/O tools or a zkVM proof π_i^{vm} for computation tools (verification predicates given in §4.3);
4. *Binding*: the requests, programs, responses, and outputs carried by the subproofs equal the canonical encodings of the corresponding spans of tr.

Tool taxonomy. zkAgent's two-class tool taxonomy follows the threat model of §2.6, which targets *provider-hosted* tools (client-side tools are trusted and excluded). Provider-hosted tools split into two classes whose integrity reduces to distinct guarantees, as stated in §4.3: *(i) networked API tools* (e.g., web search and other REST/RPC endpoints [6]), where integrity reduces to response authenticity under a designated endpoint and is realized by a zkTLS subproof; and *(ii) computation tools* (e.g., Python sandboxes [53,31]), where integrity reduces to execution correctness under a deterministic machine model and is realized by a zkVM subproof.

H Peak-Compute Estimate

This appendix details the $\sim 13\times$ peak-compute gap quoted in §5.3 between the A100 GPU on which zkLLM is measured and our CPU testbed.

Why field-multiplication throughput, not FP32 or TF32. zkLLM [69] carry out their arithmetic in the BLS12-381 scalar field, whose elements are $\sim$ 256-bit integers. Inspecting zkLLM's reference implementation, each scalar is represented as 8 limbs of 32 bits, and field multiplication is implemented entirely using 32-bit integer multiply-add and carry operations. Because A100 Tensor Cores do not support INT32 operands, zkLLM executes these limb-level operations on the CUDA-core integer pipelines rather than on Tensor Cores [50]. CPU-side, our implementation uses the `mcl` library, whose optimized x86-64 path represents approximately 256-bit field elements as 4 limbs of 64 bits and performs Montgomery multiplication over this representation [49]. Our implementation uses this scalar path and does not batch field multiplications through AVX-512 SIMD. Floating-point execution units do not participate in either implementation. Consequently, the appropriate metric is *field-multiplications per second*, derived from each platform's native integer instruction throughput and Montgomery cost, not FP32 or TF32 TFLOPS.

Departure from prior work. Prior work [60] positions the A100 and CPU comparison using the A100's TF32 Tensor-Core peak (312 TFLOPS) against a CPU estimate (10.2 TFLOPS), yielding a $\sim 30\times$ ratio. The A100 figure assumes 2:4 structured sparsity (its dense TF32 peak is 156 TFLOPS), and neither TF32 nor CPU floating-point throughput directly represents the field-arithmetic paths used by the two systems. We therefore use this comparison only as evidence of the A100's substantially greater parallel compute capacity and derive a more workload-aligned, back-of-the-envelope estimate below.

GPU side. The A100 contains 64 INT32 execution lanes per SM, 108 SMs, and a peak clock frequency of approximately 1.41,GHz [50]. As a coarse upper-bound proxy, assuming that each lane issues one 32-bit integer limb operation per cycle gives

$$64 \times 108 \times 1.41 \text{ GHz} \approx 9.75 \times 10^{12} \text{ integer limb-operations/s.}$$

In zkLLM's CUDA field-multiplication path, an eight-limb multiplication performs approximately 128 low/high multiply-add operations for the full product, another 128 for Montgomery reduction, and eight additional limb multiplications for the reduction factors, before accounting for carry propagation and final modular correction. Using the resulting 264 multiplication-related limb operations gives the optimistic estimate

$$9.75 \times 10^{12}/264 \approx 36.9 \text{ G field-multiplications/s.}$$

CPU side. Our testbed is an Intel Xeon 6982P-C (32 cores at 3.20 GHz). For the four-limb scalar representation used by `mcl` on x86-64, a CIOS-style Montgomery multiplication requires approximately $2n^2 + n = 36$ 64-bit limb multiplications for $n = 4$. As a coarse upper-bound proxy, assuming that each core issues one 64-bit limb multiplication per cycle and that the workload scales perfectly across all 32 cores gives

$$32 \times 3.20 \times 10^9/36 \approx 2.84 \text{ G field-multiplications/s.}$$

Ratio. Dividing the two idealized estimates gives

$$36.9/2.84 \approx 13,$$

the $\sim 13\times$ figure quoted in the main text. We note that this estimate would be substantially smaller had we engaged mcl's vectorized AVX-512 IFMA path. Even without this optimization, however, `zkAgent` with the Shout backend remains $53.5\times$ faster to prove and $98.9\times$ faster to verify at $T = 512$, indicating that algorithmic improvements rather than hardware advantage drive the comparison.

PROTOCOL 2 ([84]) *GKR Protocol for General Arithmetic Circuits*

Let C be a depth-d general arithmetic circuit as in Appendix C.3. The prover $\mathcal{P}$ wants to convince the verifier $\mathcal{V}$ that $\mathsf{out} = C(\mathsf{in})$. We assume $|\mathsf{out}|$ and $|\mathsf{in}|$ are powers of 2 (padding if necessary). Let $\tilde{V}_i$ and $\tilde{V}_{i,j}$ denote the MLEs defined in Appendix C.3, with $\tilde{V}_0$ and $\tilde{V}_d$ being the MLEs of out and in, respectively.

1. **Initialization.** $\mathcal{V}$ samples a random point $g^{(0)} \in \mathbb{F}^{s_0}$, computes or queries an oracle for $\tilde{V}_0(g^{(0)})$ from the claimed output, and sends $g^{(0)}$ to $\mathcal{P}$.
2. **Output-layer check (layer 0).** $\mathcal{P}$ and $\mathcal{V}$ run the sumcheck protocol to verify Eq. (33) for $i = 0$ at point $g^{(0)}$. At the end of sumcheck, $\mathcal{V}$ obtains a constant number of random points at which it needs evaluations of $\tilde{V}_{0,1}$ and $\tilde{V}_{0,j}$ (for $j > 1$). If the final sumcheck identity fails, $\mathcal{V}$ outputs rej.
3. **For each layer $i = 1, \ldots, d$:**
   (a) **Reducing claims targeting layer i.** By the end of the previous steps, $\mathcal{V}$ holds claims of the form $\widetilde{V}_{k,i}(r^{(k,i)})$ for a set of source layers $\mathcal{K}_i \subseteq \{0, \ldots, i-1\}$, and possibly an additional claim $\widetilde{V}_{i-1,i}(r^{(i-1,i)'})$ from the adjacent layer. The verifier samples random coefficients
   $$\{\alpha_{k,i} \xleftarrow{\$} \mathbb{F}\}_{k \in \mathcal{K}_i} \quad \text{and, if the additional claim is present,} \quad \alpha'_{i-1,i} \xleftarrow{\$} \mathbb{F}.$$
   and sends them to $\mathcal{P}$. Then $\mathcal{P}$ and $\mathcal{V}$ run sumcheck on the selector identity
   $$\begin{aligned} &\sum_{k \in \mathcal{K}_i} \alpha_{k,i} \widetilde{V}_{k,i}(r^{(k,i)}) + \alpha'_{i-1,i} \widetilde{V}_{i-1,i}(r^{(i-1,i)'}) \\ &\quad = \sum_{x \in \{0,1\}^{s_i}} \widetilde{V}_i(x) \Bigg[\sum_{k \in \mathcal{K}_i} \alpha_{k,i} \widetilde{C}_{k,i}(r^{(k,i)}, x) \\ &\qquad + \alpha'_{i-1,i} \widetilde{C}_{i-1,i}(r^{(i-1,i)'}, x) \Bigg]. \end{aligned}$$
   If the sumcheck passes, $\mathcal{V}$ obtains a new random point $g^{(i)} \in \mathbb{F}^{s_i}$ and a claim $\tilde{V}_i(g^{(i)})$.
   (b) **Layer check (if $i < d$).** If $i < d$, $\mathcal{P}$ and $\mathcal{V}$ run sumcheck to verify Eq. (33) for this i at point $g^{(i)}$. At the end, $\mathcal{V}$ obtains the required evaluations of $\tilde{V}_{i,i+1}$ and $\tilde{V}_{i,j}$ (for $j > i+1$) at random points determined by the sumcheck transcript, and checks the final identity. If any check fails, output rej.
4. **Input-layer check.** At the end of the above, $\mathcal{V}$ holds a claim $\tilde{V}_d(g^{(d)})$ for some $g^{(d)} \in \mathbb{F}^{s_d}$. $\mathcal{V}$ computes $\tilde{V}_d(g^{(d)})$ locally from in or queries the input commitment/oracle and accepts iff it matches the claim.